第7章 I/O设备管理

7.1 I/O系统功能、模型和接口
7.2 设备和设备控制器
7.3 中断结构和中断处理程序
7.4 设备驱动程序
7.5 设备独立性软件
7.6 用户层软件
7.7 Spooling技术
7.7 缓冲区管理
7.8 磁盘存储器的性能和调度

7.1 I/O系统功能、模型和接口

- I/O设备管理的基本功能
  - 思考：设备管理需要完成哪些功能？
    - 完成用户提出的I/O请求　　总体目标
    - 正确地访问I/O设备
      - 正确的共享设备
      - 差错控制
    - 提高CPU和I/O设备的利用率　　提高I/O设备和CPU的并行操作
    - 提高I/O处理的速度　　I/O设备控制方式，缓冲技术
    - 用户方便的使用I/O设备
      - 隐藏物理设备的细节
      - 设备无关性

7.1 I/O系统功能、模型和接口

- I/O系统的层次结构和模型

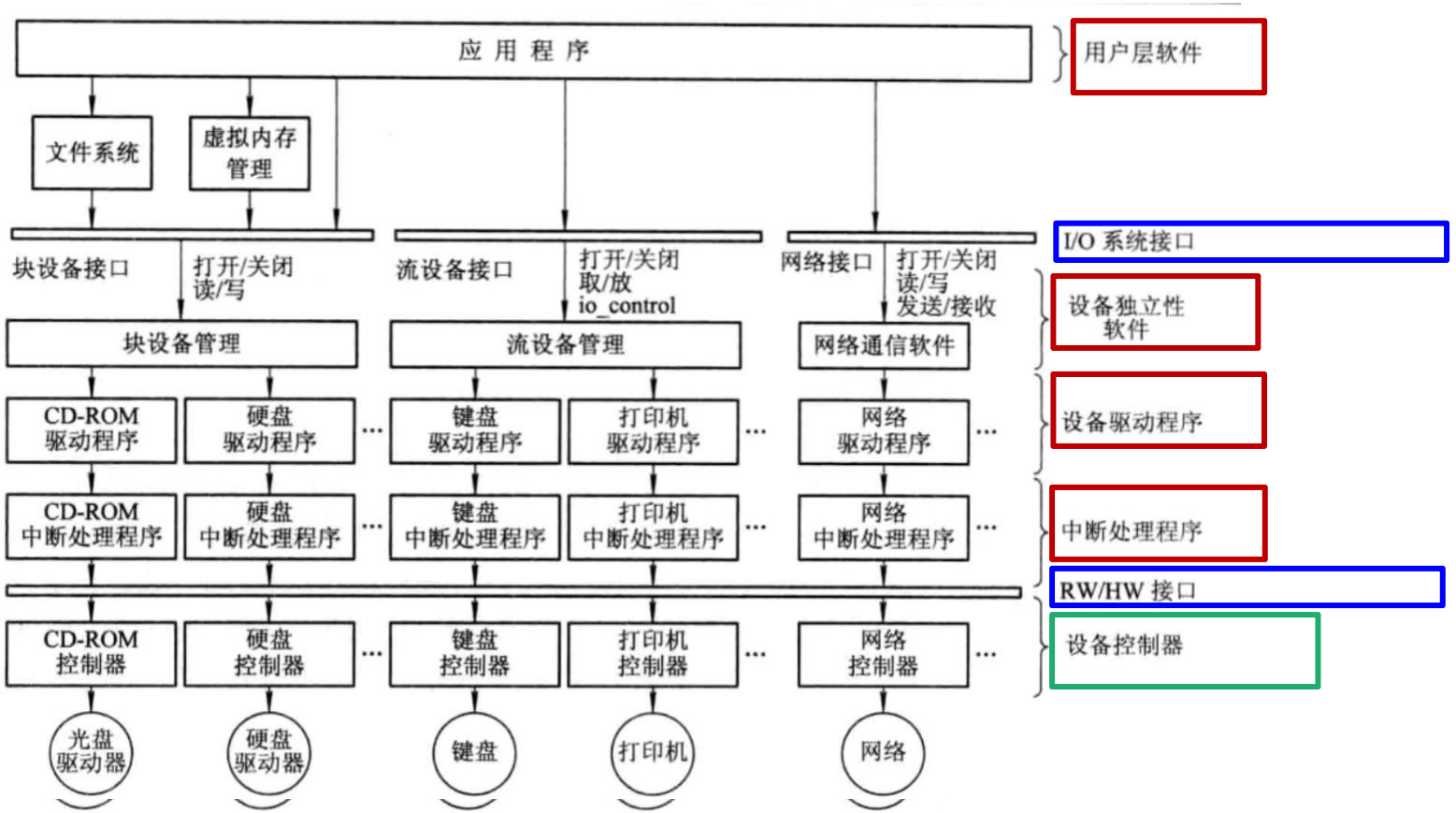


图7-1　I/O系统中各模块之间的层次视图

7.1 I/O系统功能、模型和接口

- I/O系统软件层次

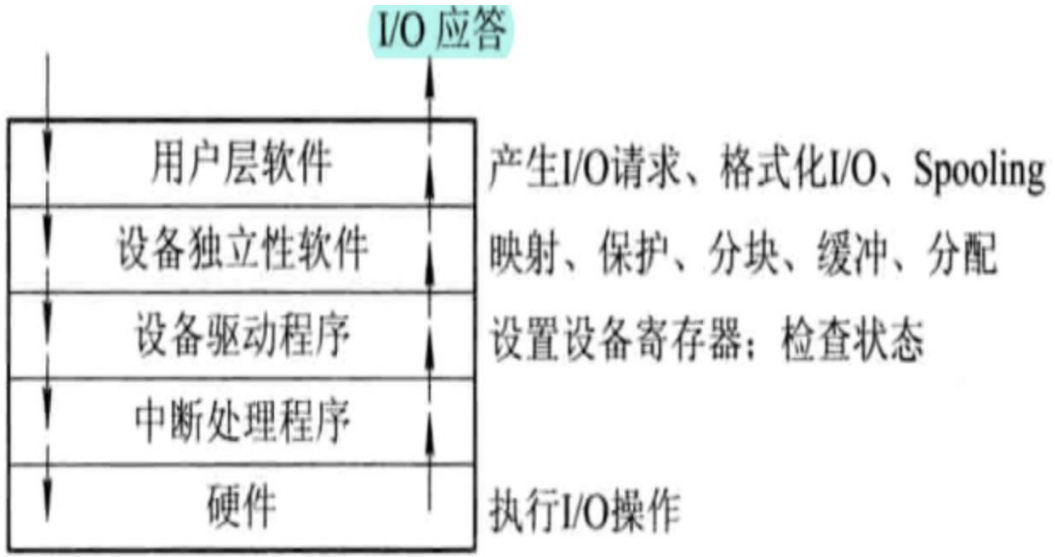


图7-2 I/O软件层次结构

7.1 I/O系统功能、模型和接口

- I/O系统接口-与上层系统之间的接口
  - 它是I/O系统与上层系统之间的接口，向上层提供对设备进行操作的抽象I/O命令，以方便高层对设备的使用。
  - 上层系统中包括有文件系统、虚拟存储器系统以及用户进程等。
  - 设备接口包括：
    - 块设备接口；流设备接口；网络通信接口

7.1 I/O系统功能、模型和接口

- 软件/硬件(RW/HW)接口-I/O系统与设备控制器的接口
  - 该接口是软件/硬件接口，在它的上面是中断处理程序和用于不同设备的设备驱动程序。
  - 在它的下面是各种设备的控制器。如CD-ROM控制器、硬盘控制器、键盘控制器、打印机控制器、网络控制器等，它们都属于硬件。
  - 由于设备种类繁多，故该接口相当复杂。

7.1 I/O系统功能、模型和接口

- 用户层软件
  - 用户层软件是实现与用户交互的接口。用户可直接调用该层所提供的、与I/O操作有关的库函数对设备进行操作。

7.1 I/O系统功能、模型和接口

- 设备独立性软件
  - 设备独立性软件也称为与设备无关的软件。其基本含义是：I/O软件独立于具体使用的物理设备。
  - 设备独立性软件的内容包括设备命名、设备分配、设备保护、数据缓冲和数据高速缓冲一类软件等。
  - 优势：提高了I/O系统的可适应性和可扩展性。在每次增加新设备或替换老设备时，都不需要对I/O软件进行修改，这样就方便了系统的更新和扩展。

7.1 I/O系统功能、模型和接口

- 设备驱动程序
  - 设备驱动程序处于I/O系统的次底层，是进程和设备控制器之间的通信程序，其主要功能是：
    - 将上层发来的抽象I/O请求转换为对I/O设备的具体命令和参数，并把它装入到设备控制器中的命令和参数寄存器中，或者相反。
  - 每类设备的驱动程序都不相同，故必须由设备制造厂商提供，而不是由OS设计者来设计。

7.1 I/O系统功能、模型和接口

- 中断处理程序
  - 中断处理程序处于I/O系统的底层，直接与硬件进行交互。当有I/O设备发来中断请求信号时，在中断硬件做了初步处理后，便转向中断处理程序。
  - 它首先保存被中断进程的CPU环境，然后转入相应设备的中断处理程序进行处理，在处理完成后，又恢复被中断进程的CPU环境，返回断点继续运行。

7.2 I/O设备和设备控制器

- I/O设备
  - I/O设备一般是由执行I/O操作的机械部分和执行控制I/O的电子部件组成，通常将这两部分分开。
  - 执行I/O操作的机械部分就是一般的I/O设备，执行控制I/O的电子部件则称为设备控制器。





在微型机、小型机中的控制器常称为控制卡、接口卡，在有的大、中型计算机系统中，还配置了I/O通道或I/O处理机。

7.2 I/O设备和设备控制器

- I/O设备分类
  - 按输入输出特性：
    - 输入型设备、输出型设备、存储型设备等；
  - 按信息交换单位：
    - 块设备——信息的存取以数据块为单位。如磁盘
    - 字符设备——信息的存取基本单位是字符。如键盘、显示器、、打印机等。
  - 按设备的共享属性分类
    - 独占设备 、共享设备 、虚拟设备
  - 按传输速率分类
    - 低速设备、中速设备、高速设备等

7.2 I/O设备和设备控制器

- 设备与设备控制器的接口
  - 通常，设备并不是直接与CPU进行通信而是与设备控制器通信。因此，在I/O设备中含有与设备控制器间的接口，在该接口中有3种类型的信号各对应一条信号线。

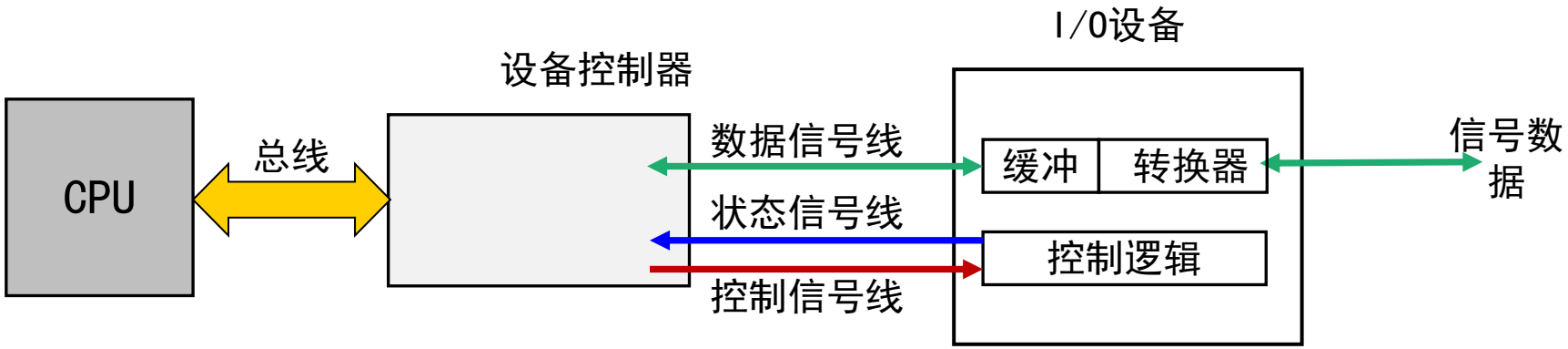
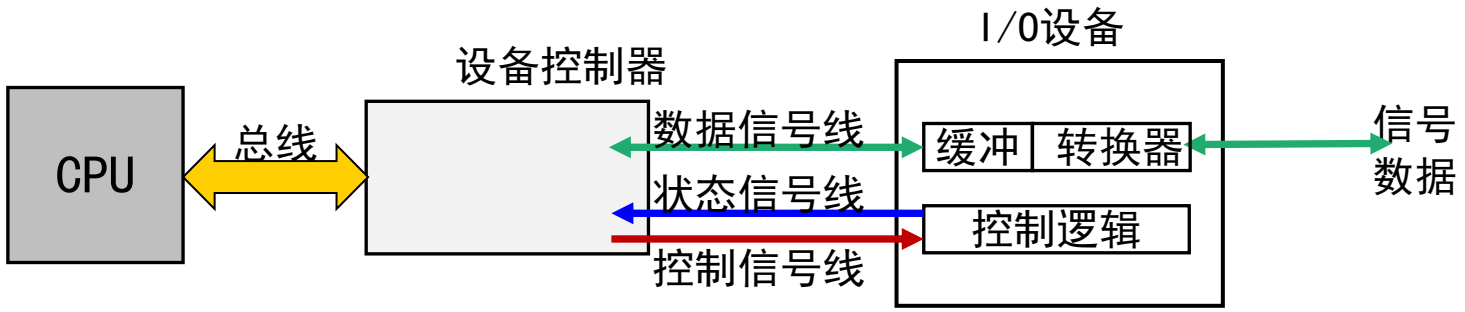


图7-3　设备与控制器间的接口

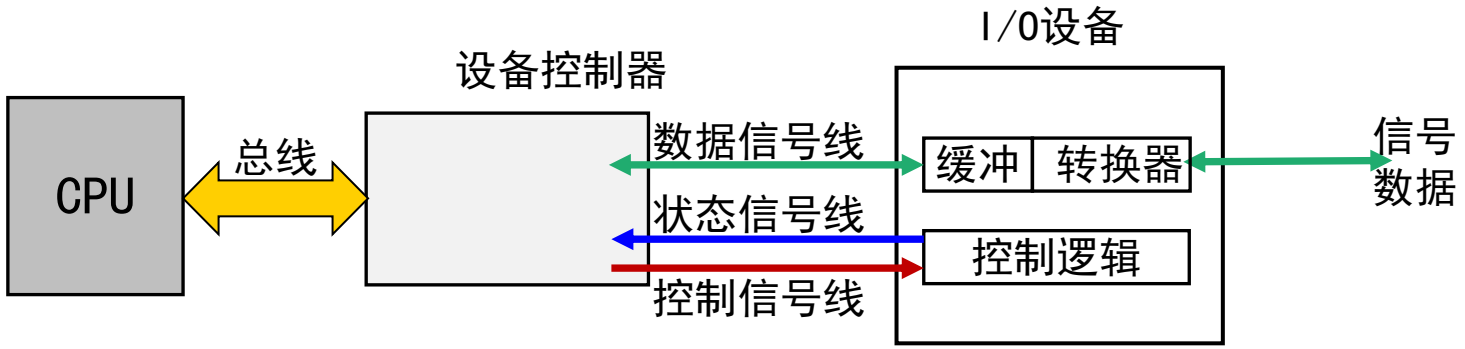
7.2 I/O设备和设备控制器

- 设备与设备控制器的接口
  - 数据信号线：用于在设备和设备控制器之间传送数据信号。
    - 对于输入设备：由外界输入的信号经转换器转换后，所形成的数据通常先送入缓冲器中，当数据量达到一定的比特(字符)数后，再从缓冲器通过一组数据信号线传送给设备控制器。
    - 对于输出设备：将从设备控制器经过数据信号线传送来的一批数据先暂存于缓冲器中，经转换器作适当转换后，再逐个字符地输出。



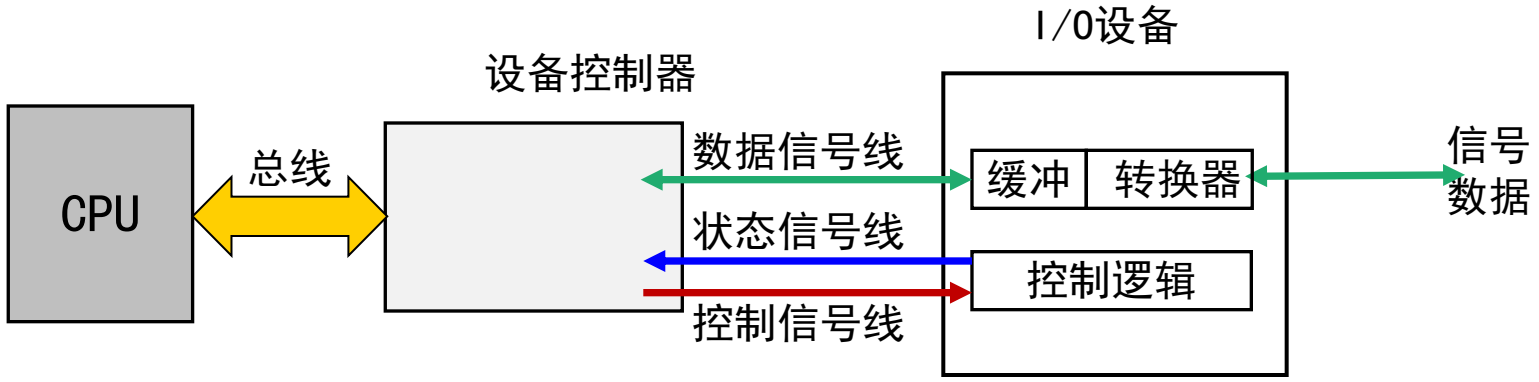
7.2 I/O设备和设备控制器

- 设备与设备控制器的接口
  - 控制信号线：
    - 由设备控制器向I/O设备发送控制信号时的通路。
    - 该信号规定了设备将要执行的操作，如读操作(指由设备向控制器传送数据)或写操作(从控制器接收数据)，或执行磁头移动等操作。



7.2 I/O设备和设备控制器

- 设备与设备控制器的接口
  - 状态信号线：
    - 该信号线用于传送指示设备当前状态的信号。
    - 设备的当前状态有正在读(或写)；设备已读(写)完成，并准备好新的数据传送。



7.2 I/O设备和设备控制器

- 设备控制器
  - 作用：控制一个或多个I/O设备，以实现I/O设备和计算机之间的数据交换。
  - 设备控制器是CPU与I/O设备之间的接口。设备控制器接收从CPU发来的命令，去控制I/O设备工作。
  - 设备控制器是一个可编址的设备
    - 仅控制一个设备时，它只有一个唯一的设备地址；
    - 若控制器可连接多个设备，则含有多个设备地址；

7.2 I/O设备和设备控制器

- 设备控制器的组成

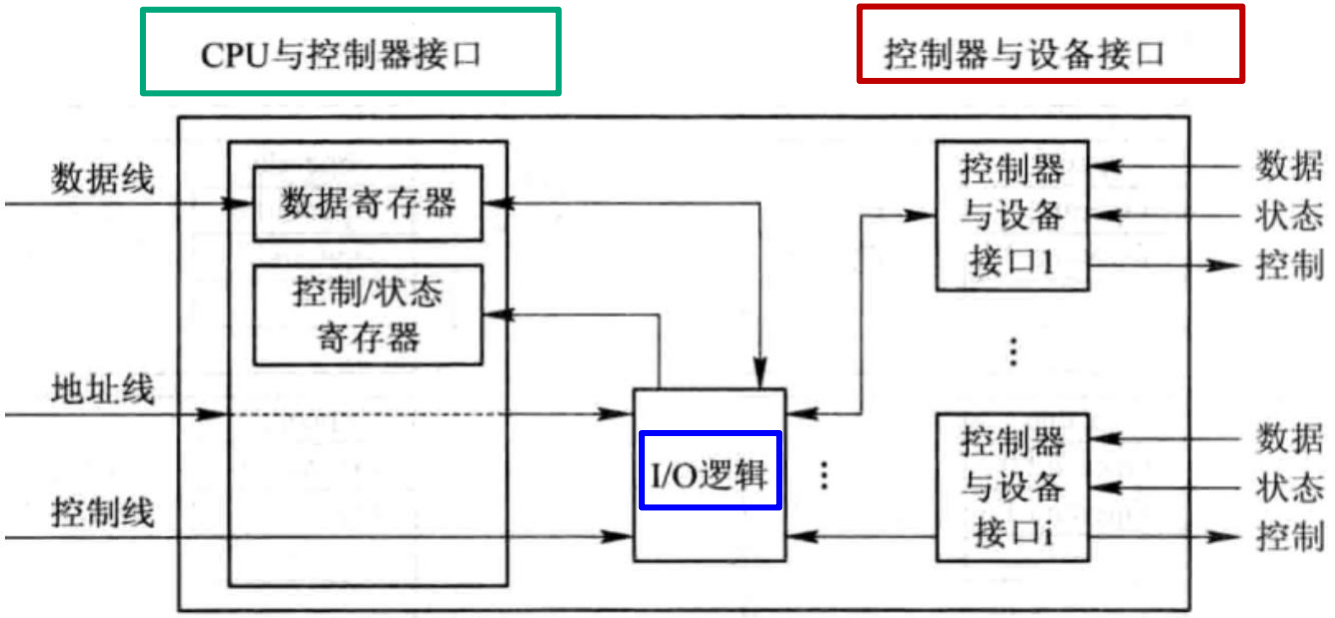
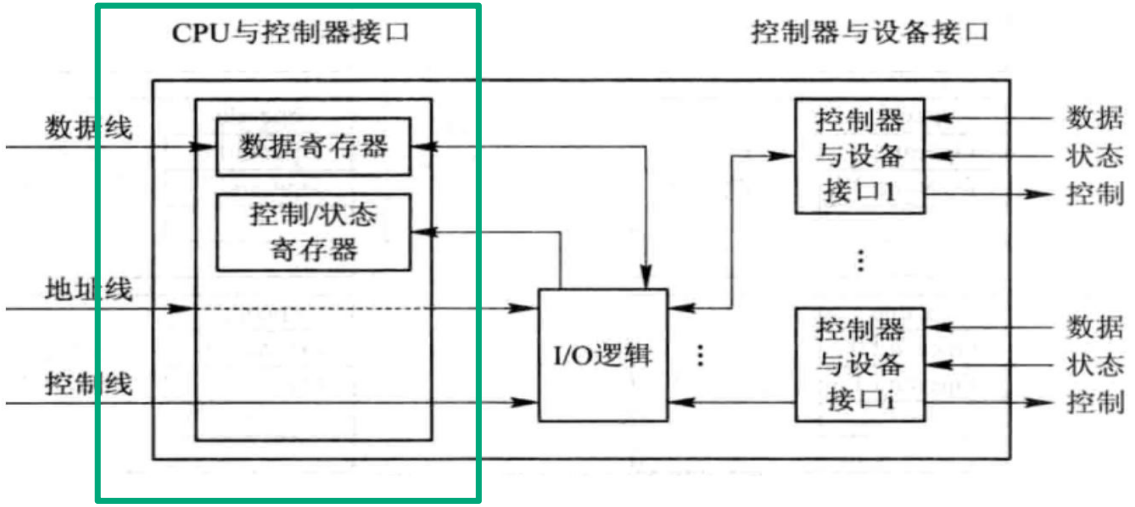


图7-4　设备控制器的组成

- CPU与控制器接口
- I/O逻辑
- 控制器于设备接口

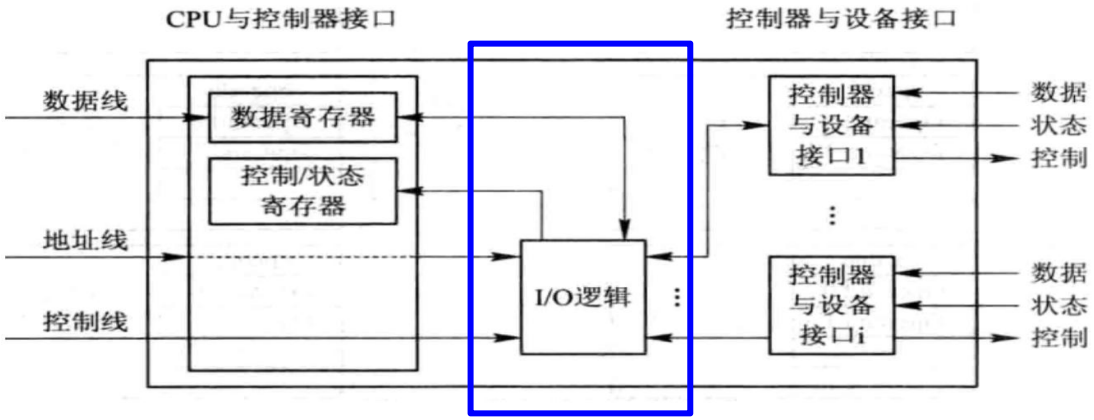
7.2 I/O设备和设备控制器

- 设备控制器的组成-CPU与设备控制器的接口
  - 该接口用于实现CPU与设备控制器之间的通信，在该接口中共有三类信号线:数据线、地址线和控制线。
  - 数据线通常与数据寄存器和控制/状态寄存器器相连接。



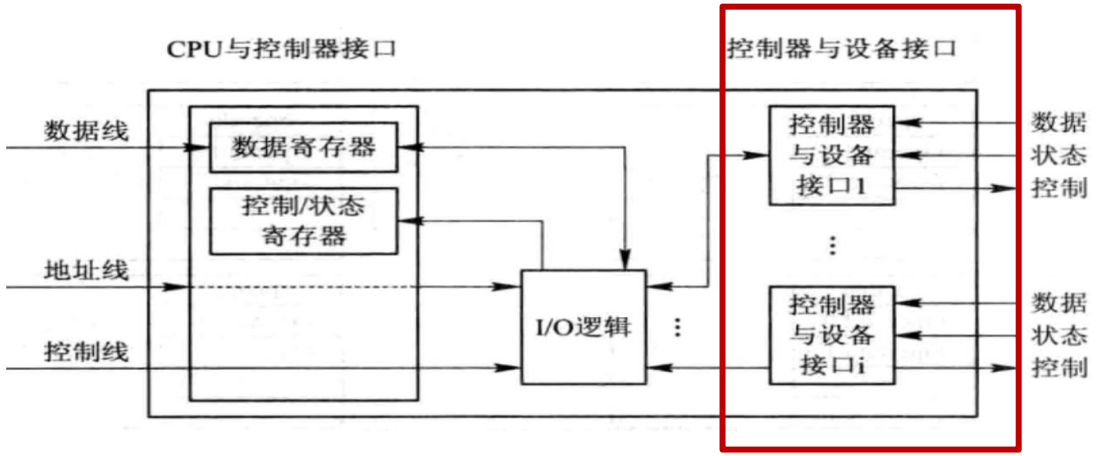
7.2 I/O设备和设备控制器

- 设备控制器的组成-I/O逻辑
  - IO逻辑用于实现对设备的控制。它通过一组控制线与处理机交互，处理机利用该逻辑向控制器发送IO命令。
  - 每当CPU要启动一个设备时，一方面将启动命令发送给控制器，另一方面又同时通过地址线把地址发送给控制器，由控制器的I/O逻辑对收到的地址译码，再根据所译出的命令对所选设备进行控制。



7.2 I/O设备和设备控制器

- 设备控制器的组成-设备控制器与设备的接口
  - 在一个设备控制器上，可以连接一个或多个设备。相应的，在控制器中有一个或多个设备接口。
  - 在每个接口中都存在数据、控制和状态三种类型的信号。
  - 控制器中的I/O逻辑根据处理机发来的地址信号去选择一个设备接口。



7.2 I/O设备和设备控制器

- 设备控制器的基本功能
  - 接收和识别命令
  - 数据交换
  - 标识和报告设备的状态
  - 地址识别
  - 数据缓冲区
  - 差错控制

7.2 I/O设备和设备控制器

- 设备控制器的基本功能
  - 接收和识别命令
    - 设备控制器能接收并识别处理机发来的多种命令。
    - 在控制器中具有相应的控制寄存器，用来存放接收的命令和参数，并对所接收的命令进行译码。
    - 例如，磁盘控制器可以接收CPU发来的read、write,format等15条不同的命令，而且有些命令还带有参数。相应地，在磁盘控制器中有多个寄存器和命令译码器等。

7.2 I/O设备和设备控制器

- 设备控制器的基本功能
  - 数据交换
    - 设备控制器可实现CPU与控制器之间、控制器与设备之间的数据交换。
    - CPU与控制器之间是通过数据总线，由CPU并行地把数据写入控制器，或从控制器中并行地读出数据。
    - 控制器与设备之间是设备将数据输入到控制器，或从控制器传送给设备。
    - 为此，设备控制器中需设置有数据寄存器。

7.2 I/O设备和设备控制器

- 设备控制器的基本功能
  - 标识和报告设备的状态
    - 控制器应记下设备的状态供CPU了解。
    - 例如，仅当该设备处于发送就绪状态时，CPU才能启动控制器从设备中读出数据。
    - 为此，在控制器中应设置一状态寄存器，用其中的每一位反映设备的某一种状态。当CPU将该寄存器的内容读入后，便可了解该设备的状态。

7.2 I/O设备和设备控制器

- 设备控制器的基本功能
  - 设备地址识别
    - 系统中的每一个设备也都有一个地址。设备控制器必须能够识别其所控制的每个设备的地址。
    - 此外，为使CPU能向(或从)寄存器中写入(或读出)数据，这些寄存器都应具有唯一的地址。控制器应能正确识别这些地址。
    - 为此，在控制器中应配置地址译码器。

7.2 I/O设备和设备控制器

- 设备控制器的基本功能
  - 数据缓冲区
    - 由于I/O设备的速率较低，而CPU和内存的速率却很高，故在控制器中必须设置一缓冲区。
    - 在输出时，用此缓冲区暂存由主机高速传来的数据，然后才以与I/O设备所匹配的速率将缓冲器中的数据传送给I/O设备。
    - 在输入时，缓冲区则用于暂存从I/O设备送来的数据，待接收到一批数据后，再将缓冲区中的数据高速地传送给主机。

7.2 I/O设备和设备控制器

- 设备控制器的基本功能
  - 差错控制
    - 对于由I/O设备传送来的数据，设备控制器还兼管进行差错检测。
    - 若发现传送中出现了错误，通常是将差错检测码置位，并向CPU报告，于是CPU将本次传送来的数据作废，并重新进行一次传送。这样便可保证数据输入的正确性。

7.2 I/O设备和设备控制器

- I/O通道（I/O　Channel）
  - 引入：在CPU和设备控制器之间增设I/O通道，目的是建立独立的I/O操作，不仅使数据的传送独立于CPU，同时对I/O操作的组织、管理及结束处理尽量独立，使一些原来有CPU处理的I/O任务转由通道来承担，以保证CPU有更多的时间进行数据处理。
  - I/O通道是一种特殊的处理机，它具有执行I/O指令的能力，并且通过执行通道程序来控制I/O操作。
  - 与一般处理机不同：指令类型单一（I/O操作指令）；通道与CPU共享内存。

7.2 I/O设备和设备控制器

- I/O通道类型
  - 字节多路通道：一种简单的共享通道，主要用于连接控制多台低速外设，以字节交叉方式传送数据。

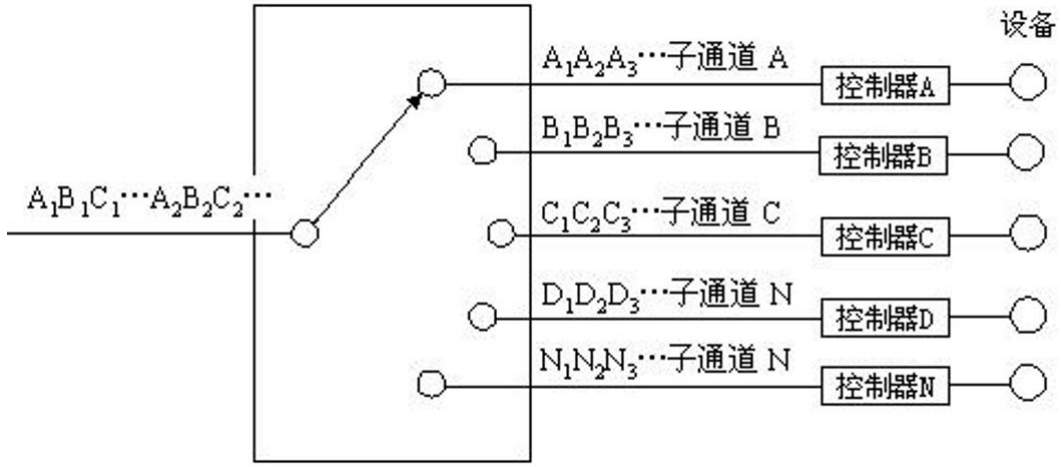


图7-5　字节多路通道

7.2 I/O设备和设备控制器

- I/O通道类型
  - 数组选择通道：
    - 又称高速通道，按数组方式进行数据传送。
    - 在物理上它可以连接多个设备，但是这些设备不能同时工作，在某一个时间段内只能执行一道通道程序，控制一台设备进行数据传送。

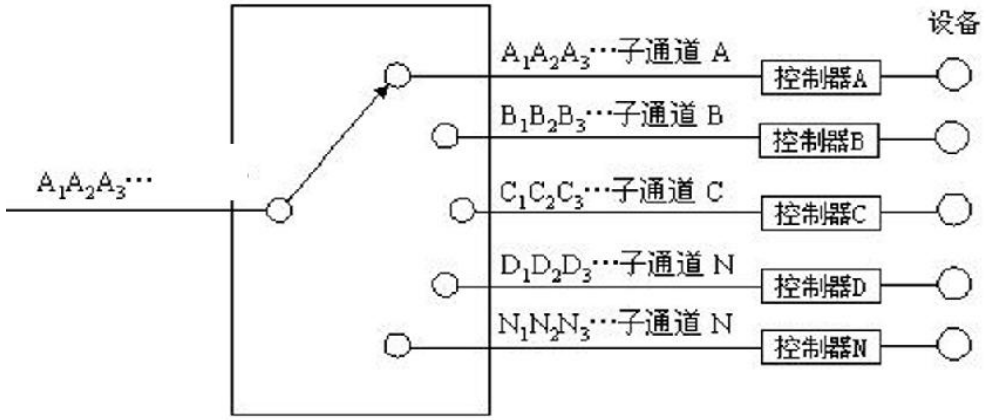


图7-6　数组选择通道

7.2 I/O设备和设备控制器

- I/O通道类型
  - 数组多路通道：
    - 连接控制多个高速外设并以成组交叉方式传送数据。
    - 某个设备进行数据传送时，通道只为该设备提供服务；当设备在执行寻址等控制性动作时，通道暂时断开与该设备的连接，挂起该设备的通道程序，而转去为其他设备提供服务，执行其他设备的通道程序。

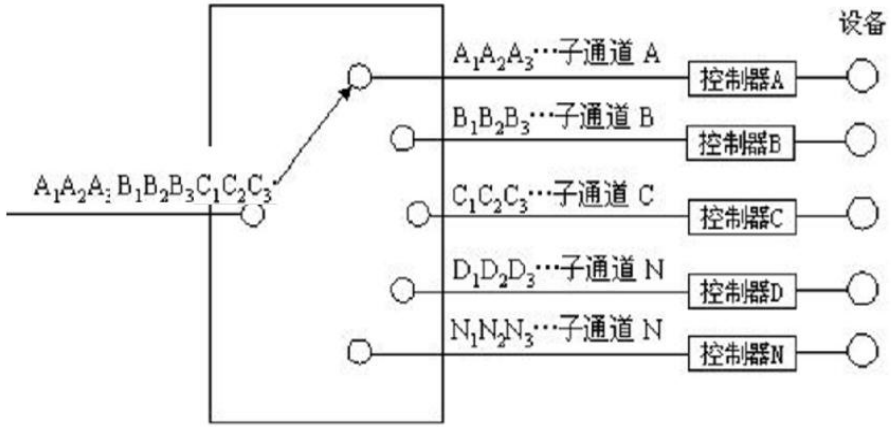


图7-7　数组多路通道

7.3 中断机构和中断处理程序

- 中断
  - 中断(interrupt) 是指CPU对I/O设备发来的中断信号的一种响应。
  - 中断发生后，CPU暂停正在执行的程序，保留CPU环境后，自动转去执行该I/O设备的中断处理程序。执行完后，再回到断点，继续执行原来的程序。
  - 这种中断是由外部设备引起的，一般称为外中断。

7.3 中断机构和中断处理程序

- 陷入
  - 陷入（Trap）由CPU内部事件引起的中断，如除零，越界访问等。
  - 与中断一样，若系统发现了有陷入事件，CPU也将暂停正在执行的程度，转去执行该陷入事件的处理程序。
  - 陷入的中断事件来源于内部，通常称为内中断。

7.3 中断机构和中断处理程序

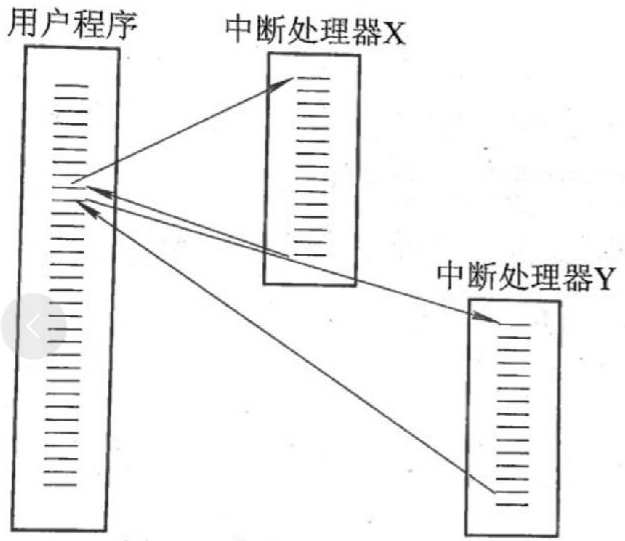
- 中断向量表
  - 为每个设备的中断请求规定一个中断号，直接对应于中断向量表的一个表项。
  - 为每种设备配以相应的中断处理程序，并把该程序的入口地址放在中断向量表中与该设备中断号对应的表项里。
  - I/O设备发来中断请求信号时，由中断控制器确定该请求的中断号，根据中断号查找中断向量表，取得该设备的中断处理程序的入口地址，转入中断处理程序执行。

7.3 中断机构和中断处理程序

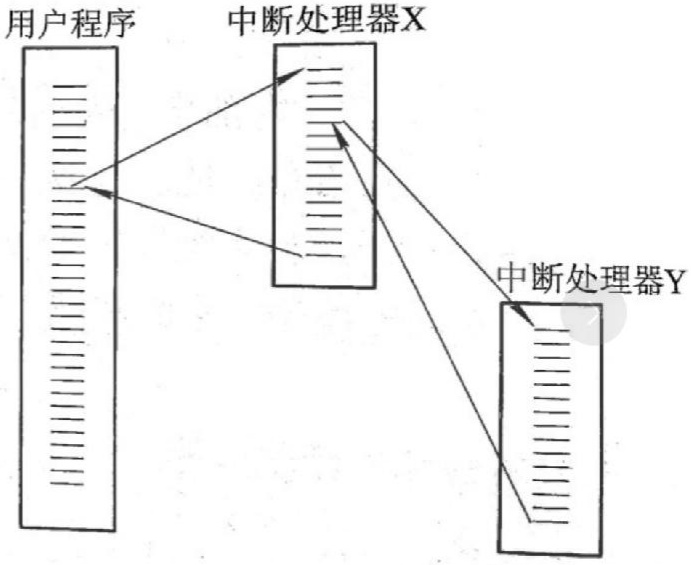
- 多中断处理
  - 屏蔽（禁止）中断：
    - 当处理机正在处理一个中断时，将屏蔽掉所有的中断
    - 直到处理机已处理完本次中断，再去检查是否有中断产生。若有，则处理被挂起的中断；若无，则返回被中断的程序。所有中断按顺序处理。
    - 特点：简单，但不能用于实时性要求较高的中断请求。
  - 嵌套中断：
    - 在设置了中断优先级的系统中，当同时有多个不同优先级的中断请求，CPU优先响应优先级最高的中断请求
    - 高优先级的中断请求可以抢占正在运行的低优先级中断的处理机。

7.3 中断机构和中断处理程序

- 多中断处理



（a）顺序中断处理



（b）嵌套中断处理

图7-8 多中断处理方式

7.3 中断机构和中断处理程序

- 中断处理程序
  - 中断处理程序的处理步骤
    - 1、测定是否有未响应的中断信号
    - 2、保护被中断进程的CPU环境
    - 3、转入相应的设备中断处理程序
    - 4、中断处理
    - 5、恢复被中断进程CPU的现场并退出中断

7.3 中断机构和中断处理程序

- 中断处理程序

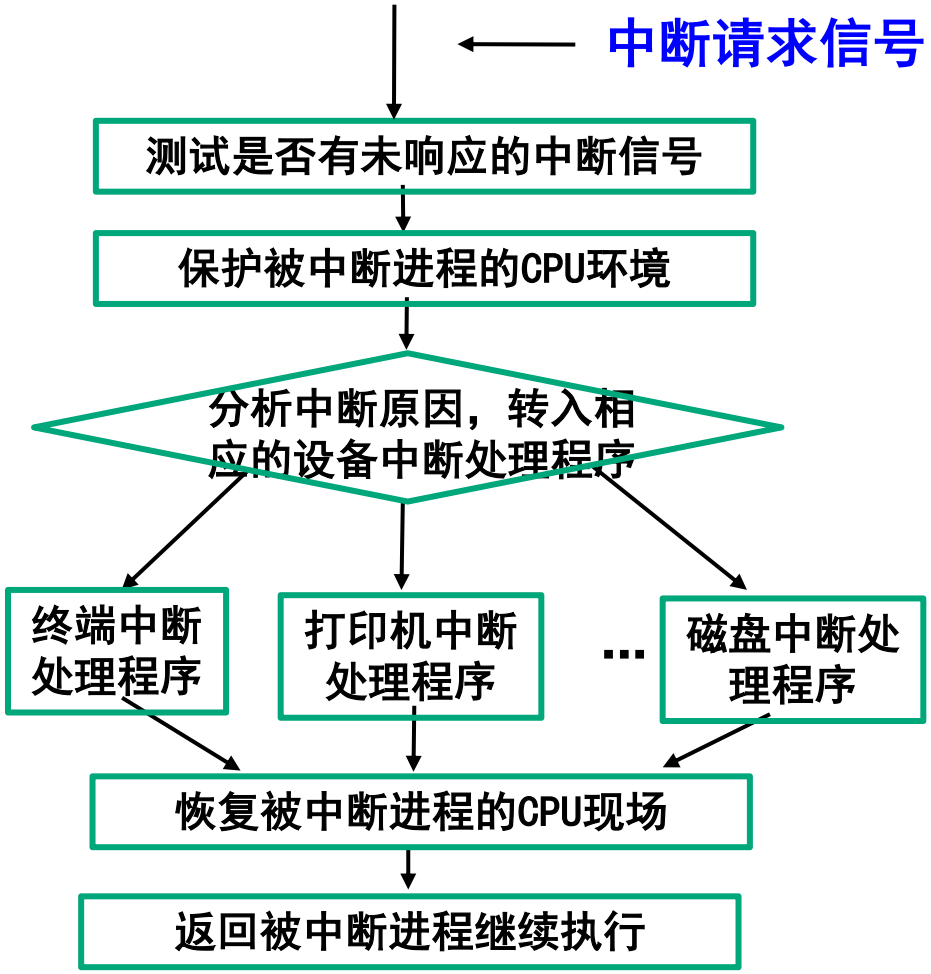


图7-10 中断处理流程

7.3 中断机构和中断处理程序

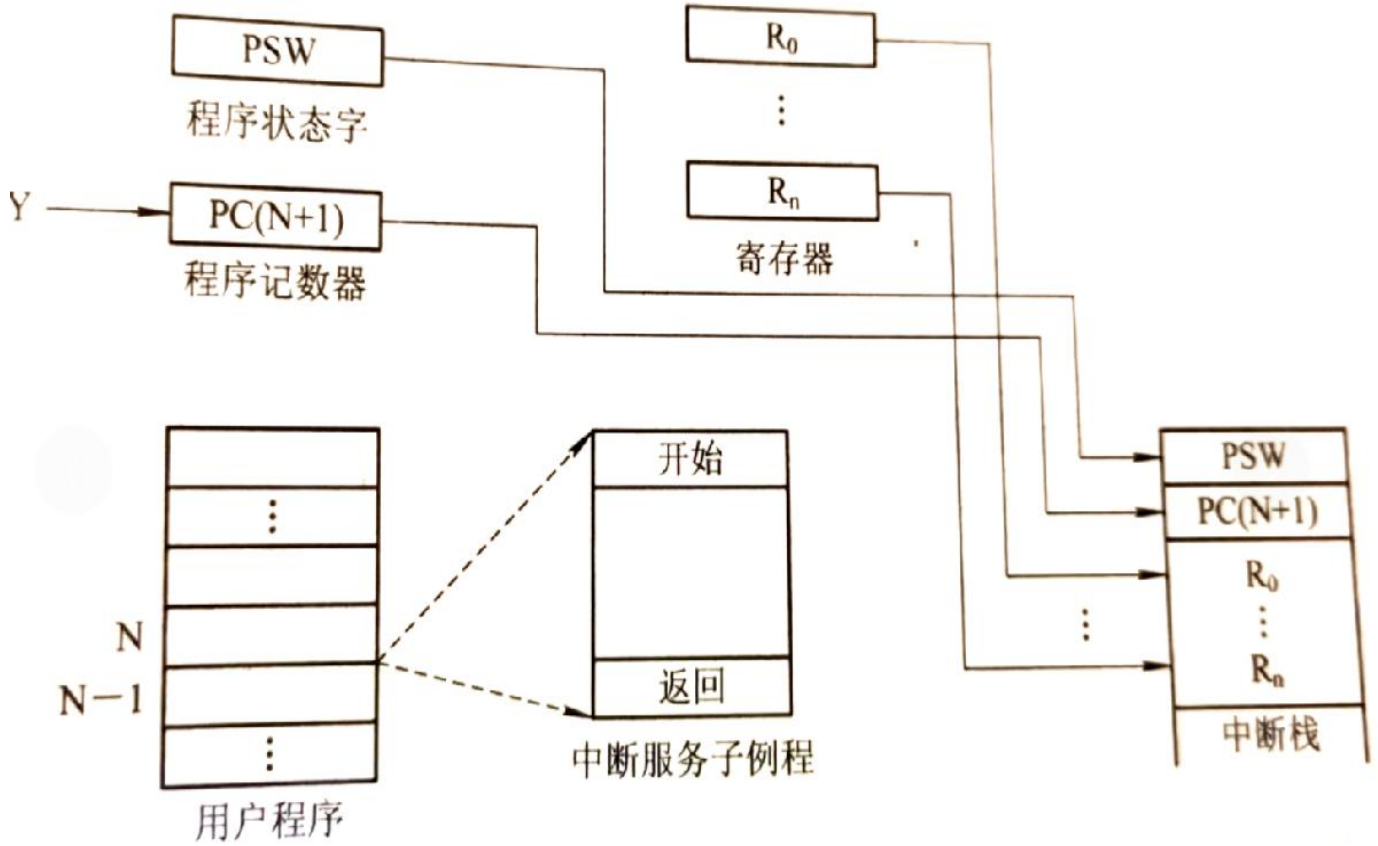
- 中断处理程序
  - 1、测定是否有未响应的中断信号
    - 每当设备完成一个字符（字或数据块）的读入（或输出），设备控制器便向处理机发送一个中断请求信号。请求处理机将设备已读入的数据传送到内存的缓冲区中（读入），或者请求处理机将要输出的数据（输出）传送给设备控制器。
    - 程序每当执行完当前指令后，处理机都要测试是否有未响应的中断信号。若没有，继续执行下一条指令。若有，则停止原有进程的执行，准备转去执行中断处理程序。

7.3 中断机构和中断处理程序

- 中断处理程序
  - 2、保护被中断进程的CPU环境
    - 在把控制权转交给中断处理程序之前，需要先保护被中断进程的CPU 环境，以便以后能恢复运行。
    - 首先由硬件自动将程序状态字寄存器PSW和保存在程序计数器PC中下一条指令的地址保存在中断保留区（栈）中。
    - 然后，把被中断进程的CPU现场信息，即将包括所有 CPU 寄存器的（如通用寄存器、段寄存器等）内容都压入中断栈中。

7.3 中断机构和中断处理程序

- 中断处理程序
  - 2、保护被中断进程的CPU环境



7.3 中断机构和中断处理程序

- 中断处理程序
  - **3、转入相应的设备中断处理程序**
    - 由处理机对各个中断源进行测试，以确定引起本次中断的I/O设备，并向提供中断信号的设备发送确认信号。
    - 该设备收到确认信号后，就立即取消它所发出的中断请求信号。
    - 然后，将相应的设备中断处理程序的入口地址装入到程序计数器中。这样，当处理机运行时，便可自动地转向设备中断处理程序。

7.3 中断机构和中断处理程序

- 中断处理程序
  - 4、中断处理
    - 对不同的设备，有不同的中断处理程序。该程序首先从设备控制器中读出设备状态，以判别本次中断是正常完成中断还是异常结束中断。
    - 若是正常中断，中断程序便做结束处理。若是异常结束中断，则根据发生异常的原因做相应的处理。

7.3 中断机构和中断处理程序

- 中断处理程序
  - 5、恢复被中断进程的现场
    - 中断处理完成以后，需要恢复CPU的现场，退出中断。但是此刻是否返回到被中断的进程，取决于两个因素：
      - ①本中断是否采用了屏蔽中断方式，若是，就会返回被中断的进程。
      - ②采用的是中断嵌套方式，如果没有优先级更高的中断请求I/O，在中断完成后，仍会返回被中断的进程；反之，系统将处理优先级更高的中断请求。

7.3 中断机构和中断处理程序

- 中断处理程序
  - **5、恢复被中断进程的现场**
    - 如果是要返回到被中断的进程，可将保存在中断栈中的被中断进程的现场信息取出，并装入到相应的寄存器中，其中包括该程序下一次要执行的指令的地址N+1、PSW以及各通用寄存器和段寄存器的内容。
    - 当处理机再执行本程序时，便从N+1处开始，最终返回到被中断的程序。

7.4 设备驱动程序

- 设备驱动程序的功能
  - 设备驱动程序又称为设备处理程序，它是实现在设备独立性软件和设备控制器之间通信和转换的程序，具体说，它将抽象的I/O请求转换成具体的I/O操作后传送给控制器。又把控制器中所记录的设备状态和I/O操作完成情况，及时地反映给请求I/O的进程。

7.4 设备驱动程序

- 设备驱动程序的功能
  - 设备驱动程序又称为设备处理程序，它是I/O进程与设备控制器之间的通信程序（进程）。功能主要是：
  - 接收由设备独立性软件发来的命令和参数，并将命令中的抽象要求转换为具体要求。

    例如，将磁盘块号转换为磁盘的盘面、磁道号和扇区号。
  - 检查用户I/O请求的合法性、了解I/O设备状态、传递有关参数、设置设备工作方式
  - 发出I/O命令。如果设备空闲，便立即启动I/O设备去完成指定的I/O操作；如果设备处于忙碌状态，则将请求者的请求块挂在设备队列上等待。
  - 及时响应由控制器或通道来的中断请求，并根据其中断类型调用相应的中断处理程序。

7.4 设备驱动程序

- 设备驱动程序的特点
  - 驱动程序与设备控制器以及I/O设备的硬件特性紧密相关，对于不同类型的设备，应配置不同的驱动程序。但可以为相同的多个终端设置一个终端驱动程序。
  - 驱动程序与I/O设备所采用的I/O控制方式紧密相关。
  - 驱动程序与硬件紧密相关，因而其中的一部分必须用汇编语言书写。目前很多驱动程序的基本部分已经固化在ROM中。
  - 驱动程序应允许重入。一个正在运行的驱动程序常会在一次调用完成前被再次调用。
  - 驱动程序不允许系统调用。但是为了满足其与内核其他部分的交互，可以允许对某些内核过程调用。如通过调用内核过程来分配和释放内存页面做为缓冲区，以及调用其他过程来管理MMU定时器、DMA控制器、中断控制器等。

7.4 设备驱动程序

- 设备驱动程序的处理过程
  - 1）将抽象要求转换为具体要求
  - 2）检查I/0请求的合法性
  - 3）读出和检查设备的状态
  - 4）传送必要的参数
  - 5）工作方式的设置
  - 6）启动I/0设备

7.4 设备驱动程序

- 设备驱动程序的处理过程
  - 1）将抽象要求转换为具体要求
  - 通常在每个设备控制器中含有若干个寄存器，分别用于暂存命令、数据和参数等。
  - 用户和上层软件对设备控制器的具体情况毫无了解，因而只能向它发出抽象的要求(命令)，但这些命令无法传送给设备控制器。
  - 需要将这些抽象要求转换为具体要求。例如，将抽象要求中的盘块号转换为磁盘的盘面、磁道号及扇区号。这一转换工作只能由驱动程序来完成。
  - 在OS中，只有驱动程序才能同时了解抽象要求和设备控制器中的寄存器情况；也只有驱动程序才知道命令、数据和参数应分别送往哪个寄存器。

7.4 设备驱动程序

- 设备驱动程序的处理过程
  - 2）检查I/O请求的合法性
    - 对于任何I/O设备，都只能完成一组特定的功能，若该设备不支持这次的I/O请求，则认为这次I/O请求非法。
  - 3）读出和检查设备的状态
    - 在启动设备之前，要从设备控制器的状态寄存器中，读出设备的状态。例如，为了向某设备写数据，应先检查该设备是否处于接收就绪状态，仅当它处于接收就绪状态时，才能启动其设备控制器，否则只能等待。

7.4 设备驱动程序

- 设备驱动程序的处理过程
  - 4）传送必要的参数
    - 例如启动磁盘进行读/写之前，应先将本次要传送的字节数和数据应到达的内存始址，送入控制器相应寄存器中。
  - 5）工作方式的设置
    - 有些设备可具有多种工作方式，典型情况是利用RS232接口进行异步通信。在启动该接口之前，应先按通信规程设定参数：波特率、奇偶校验方式、停止位数目及数据字节程度等。

7.4 设备驱动程序

- 设备驱动程序的处理过程
  - 6）**启动I/O设备**
    - 在完成上述各项工作之后，驱动程序可以向控制器中的命令寄存器传送相应的命令。
    - 驱动程序发出I/O命令后，基本的I/O操作是在设备控制器的控制下进行的。
    - 通常，I/O操作所需完成的工作较多，需要一定的时间，如读/写一个盘块中的数据，此时驱动(程序)进程把自己阻塞起来，直到中断到来时才将它唤醒。

7.5 设备独立性软件

- 设备独立性的概念
  - 为了提高OS的可适应性和可扩展性，现代操作系统都实现了设备独立性(Device Independence)，也称为设备无关性。其基本含义：应用程序独立于具体使用的物理设备。
  - 为了实现设备独立性，必须再在驱动程序之上设置一层软件，称为设备独立性软件。其基本功能是执行对所有设备公共的I/O功能，并且向用户层软件提供一个统一的接口。
  - 优势：设备分配时的灵活性；易于实现I/O重定向

7.5 设备独立性软件

- 逻辑设备与物理设备
  - 为了实现设备独立性而引入了逻辑设备和物理设备的概念。
  - 在应用程序中，用逻辑设备名来请求使用某类设备；而系统实际执行时，还必须使用物理设备名。因此，需有逻辑设备名→物理设备名的转换功能。

7.5 设备独立性软件

- 设备独立性软件的功能
  - 向用户层（或文件层）提供统一接口
  - 执行所有设备的公有操作
    - 将逻辑设备名映射为物理设备名，进而可以找到相应物理设备的驱动程序
    - 对设备进行保护，禁止用户直接访问设备，防止无权访问的用户使用
    - 缓冲管理
    - 差错控制
    - 对独占设备的分配和回收
    - 提供独立于设备的大小统一的逻辑数据块

7.5 设备独立性软件

- 设备独立性软件的功能
  - 逻辑设备名到物理设备名映射的实现
    - 逻辑设备表

逻辑设备名	物理设备名	驱动程序入口地址
/dev/tty	3	1024
/dev/printer	5	2046

7.5 设备独立性软件

- 设备分配
  - 为防止诸进程对系统资源的无序竞争，设备不允许用户自行使用，必须由系统统一分配。
  - 每当进程向系统提出I/O请求时，只要是可能和安全的，设备分配程序便按一定的策略，把设备分配给请求进程。有的系统还应分配相应的控制器和通道。

7.5 设备独立性软件

- 设备分配时应考虑的因素
  - 设备的固有属性
    - 独占设备、共享设备、虚拟设备
  - 设备分配算法
    - 先来先服务 、优先级等
  - 设备分配中的安全性
    - 安全分配方式——进程提出I/O请求后即阻塞，使该进程不可能再请求任何资源，在运行时又不保持任何资源，摒弃“请求和保持”条件。
    - 不安全分配方式——发出I/O请求后仍继续运行，仅当进程所请求的设备已被另外一个进程占用，请求进程才阻塞。——具有“请求和保持”条件，有可能死锁。

7.5 设备独立性软件

- 设备分配
  - 数据结构
    - 系统设备表SDT
    - 设备控制表DCT
    - 设备控制器控制表COCT
    - 通道控制表CHCT

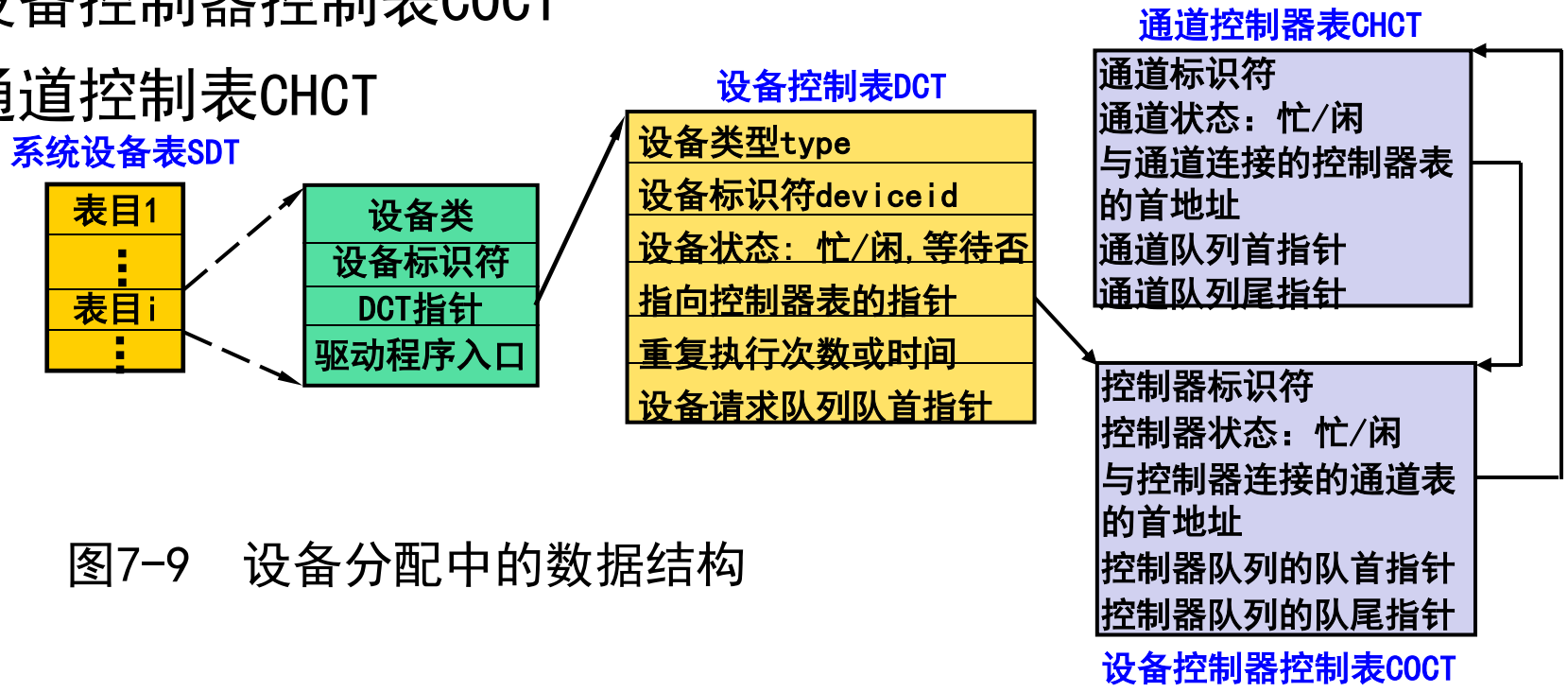


图7-9　设备分配中的数据结构

7.5 设备独立性软件

- 设备分配
- 按下述步骤进行设备分配
  - 分配设备
    - 根据I/O请求中的物理设备名查找系统设备表SDT，从中找出该设备的DCT。
    - 查询DCT中设备状态字段，若忙，将请求I/O进程的PCB挂在设备队列上；否则，按分配算法，计算本次设备分配的安全性
    - 如安全，将设备分配给请求进程；否则，仍将进程PCB插入到设备等待队列上。
  - 分配控制器
  - 分配通道

7.6 用户层I/O软件

- 用户层I/O软件
  - 系统调用
    - 引入
      - 为使诸进程能有条不紊地使用I/O设备，且能保护设备的安全性，不允许运行在用户态的应用进程去直接调用运行在核心态(系统态)的OS过程。
      - 同时应用进程在运行时，又必须取得OS所提供的服务，否则，应用程序几乎无法运行。
    - 为了解决以上两个问题，OS在用户层中引入了一个中间过程——系统调用，来对I/O设备进行操作。当应用程序需要执行某种I/O操作时， 在应用程序中必须使用相应的系统调用。

7.6 用户层I/O软件

- 用户层I/O软件
  - 系统调用
    - 当OS捕获到应用程序中的该系统调用后，便将CPU的状态从用户态转换到核心态，然后转向操作系统中相应过程，由该过程完成所需的I/O操作。
    - 执行完成后，系统又将CPU状态从核心态转换到用户态，返回到应用程序继续执行。
    - 由OS向用户提供的所有功能，用户进程都必须通过系统调用来获取，或者说，系统调用是应用程序取得OS所有服务的唯一途径。

7.6 用户层I/O软件

- 系统调用与库函数
  - 用户层软件必须通过一组系统调用来取得操作系统服务。
  - C语言等现代高级语言提供了与各系统调用一一对应的库函数，用户通过调用对应的库函数使用系统调用，如C语言中库函数write等。

7.6 用户层I/O软件

- 练习：
- 用户程序发出磁盘I/O请求后，系统的正确处理流程是（　）

A、用户程序→系统调用处理程序→中断处理程序→设备驱动程序

B、用户程序→系统调用处理程序→设备驱动程序→中断处理程序

C、用户程序→设备驱动程序→系统调用处理程序→中断处理程序

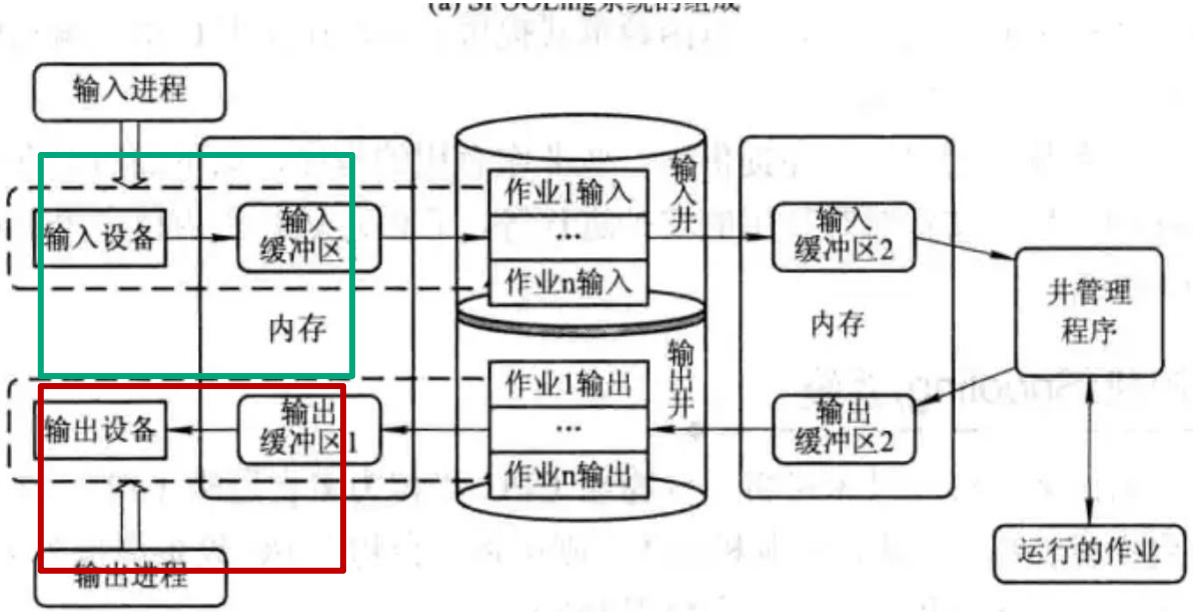
D、用户程序→设备驱动程序→中断处理程序→系统调用处理程序

7.7 Spooling技术

- Spooling技术基本概念
- 为了缓和CPU的高速性与I/O设备低速性间的矛盾而引入了脱机输入、脱机输出技术。
- 当系统中引入了多道程序技术后，利用其中的一道程序，来模拟脱机输入时的外围控制机功能，把低速I/O设备上的数据传送到高速磁盘上；用另一道程序来模拟脱机输出时外围控制机的功能，把数据从高速磁盘传送到低速输出设备上。
- 在主机的直接控制下，实现脱机输入、输出功能。外围操作与CPU对数据的处理同时进行。
- 这种在联机情况下实现的同时外围操作称为SPOOLing(Simultaneaus Periphernal Operating On-Line)技术，或称为假脱机操作。

7.7 Spooling技术

- Spooling系统的组成



- 输入井和输出井：在磁盘上开辟的两个大的存储空间。输入井模拟脱机输入时的高速磁盘，用于暂存I/O设备输入的数据。输出井模拟脱机输出时的高速磁盘，用于暂存用户程序的输出数据。

7.7 Spooling技术

- Spooling系统的组成

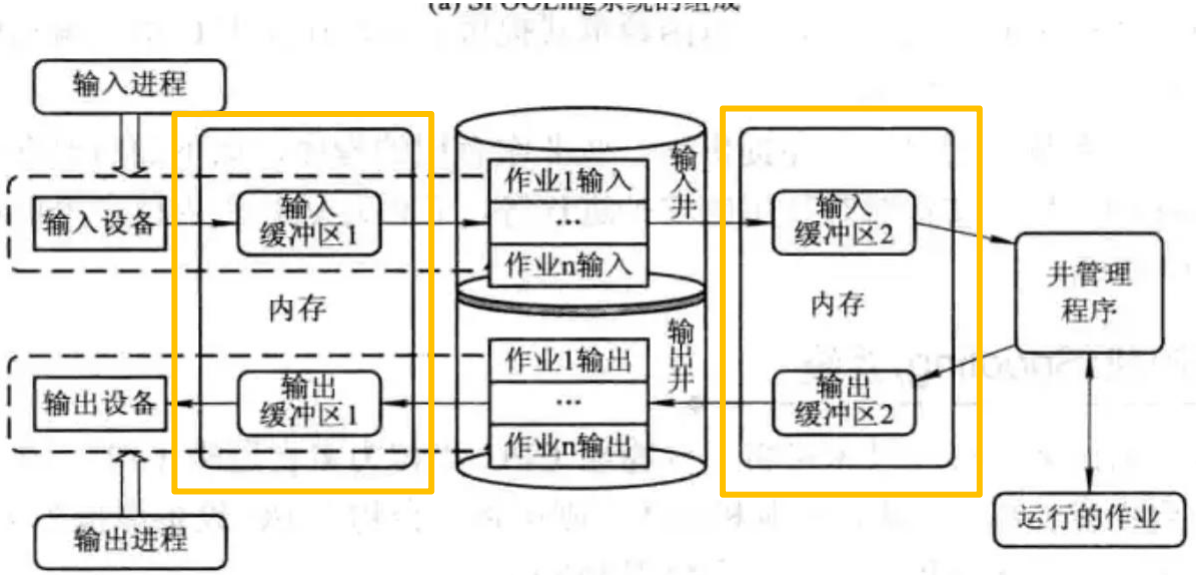
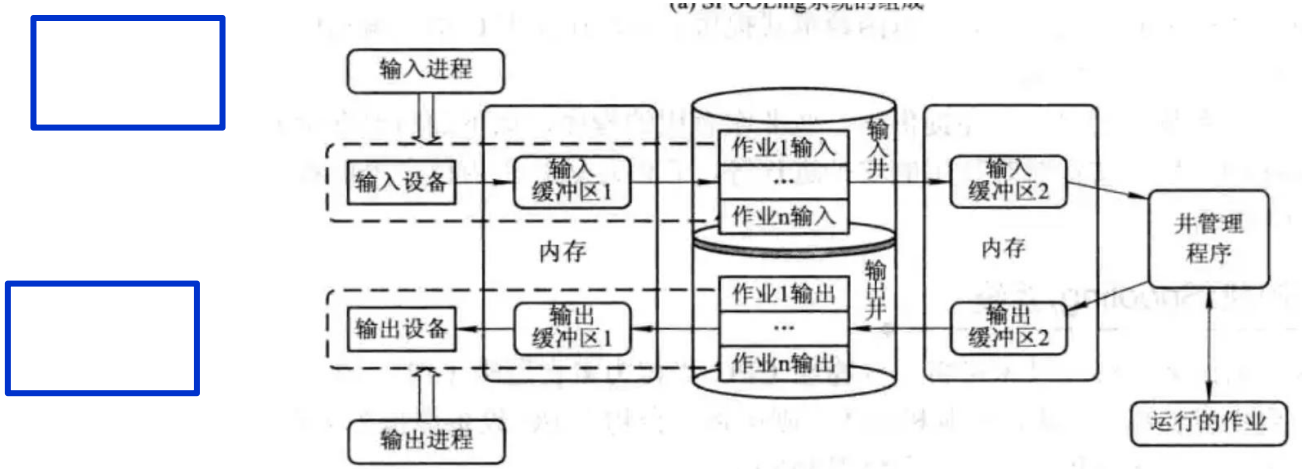


图7-10　Spooling系统的组成

- **输入缓冲区和输出缓冲区：**在内存中开辟的两个缓冲区，**输入缓冲区**用于暂存由输入设备送来的数据，再传送到输入井。**输出缓冲区**用于暂存从输出井送来的数据，再传送给输出设备。

7.7 Spooling技术

- Spooling系统的组成



- 输入进程SP_i：在输入设备空闲时，输入进程将用户要求的数据预先从输入设备通过输入缓冲区再送到输入井。当CPU需要输入数据时，直接从输入井读入内存。
- 输出进程SPo：输出进程把用户要求输出的数据，随时从内存送到输出井，待输出设备空闲时，再将输出井中的数据，经过输出缓冲区送到输出设备上。

7.7 Spooling技术

- Spooling技术的应用-虚拟打印机

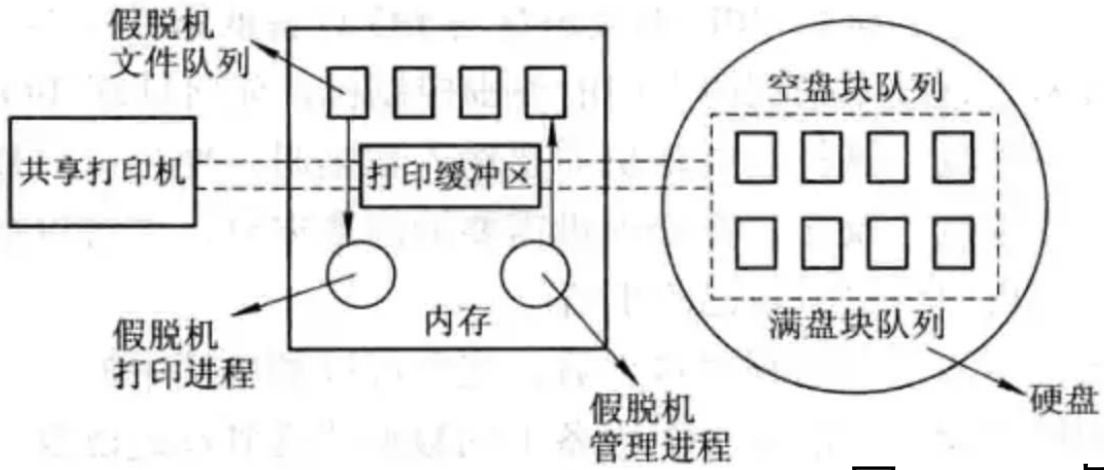
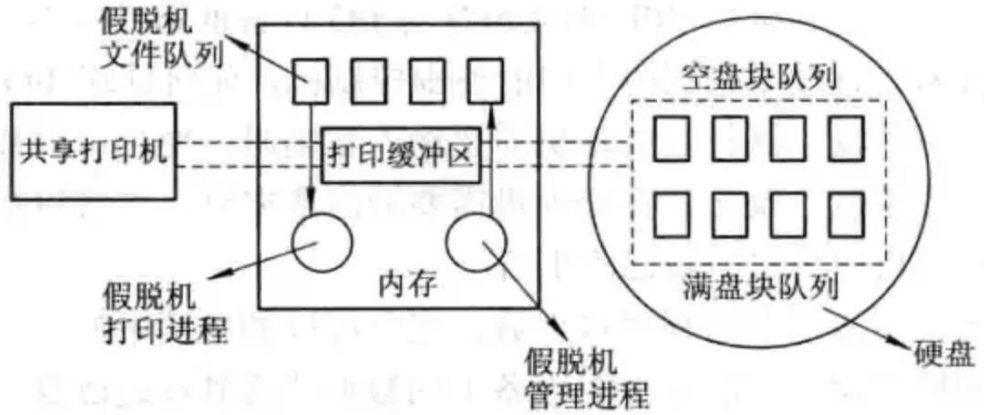


图7-10　虚拟打印机

- 假脱机管理进程
  - 接收用户进程的打印请求，将打印数据存入磁盘缓冲区（输出井），并为每个请求创建任务控制块（如打印请求表），将其加入打印任务队列。
  - 管理磁盘缓冲区资源，维护任务队列的顺序（如优先级、时间戳），确保任务按规则排队。
  - 与假脱机打印进程通信，通知其队列状态变化。

7.7 Spooling技术

- Spooling技术的应用-虚拟打印机



- 假脱机打印进程
  - 从打印任务队列中取出任务，将数据从磁盘缓冲区传输到内存缓冲区，再发送至打印机。
  - 与打印机硬件交互，处理打印过程中的状态反馈（如缺纸、卡纸）。
  - 打印完成后释放相关资源（如内存缓冲区、磁盘空间），更新任务状态。

7.7 Spooling技术

- Spooling系统的特点
  - 提高了I/O速度：对数据所进行的I/O操作，从对低速I/O设备进行的I/O操作，演变为对输入井或输出井中数据的存取。
  - 将独占设备改造成虚拟共享设备：在Spooling系统中，实际上并没有为任何进程分配物理设备，只是在输入井或输出井中为进程分配一个存储区和建立一张I/O请求表。
  - 实现了虚拟设备功能：将独占设备变换为若干台对应的逻辑设备的功能。

7.8 缓冲管理

- 引入缓冲的目的
  - 为了缓和CPU与I/O设备速度不匹配的矛盾

    到达速率与离去速率不同的地方均可设置缓冲区。
  - 减少对CPU的中断频率
  - 解决数据粒度不匹配
  - 提高CPU和I/O设备的并行性

7.8 缓冲管理

- 单缓冲
- 在块设备输入时，假定：
  - T:从磁盘把一块数据输入到缓冲区的时间
  - M:操作系统将该缓冲区数据送到用户区的时间
  - C: CPU对这块数据处理（计算）时间

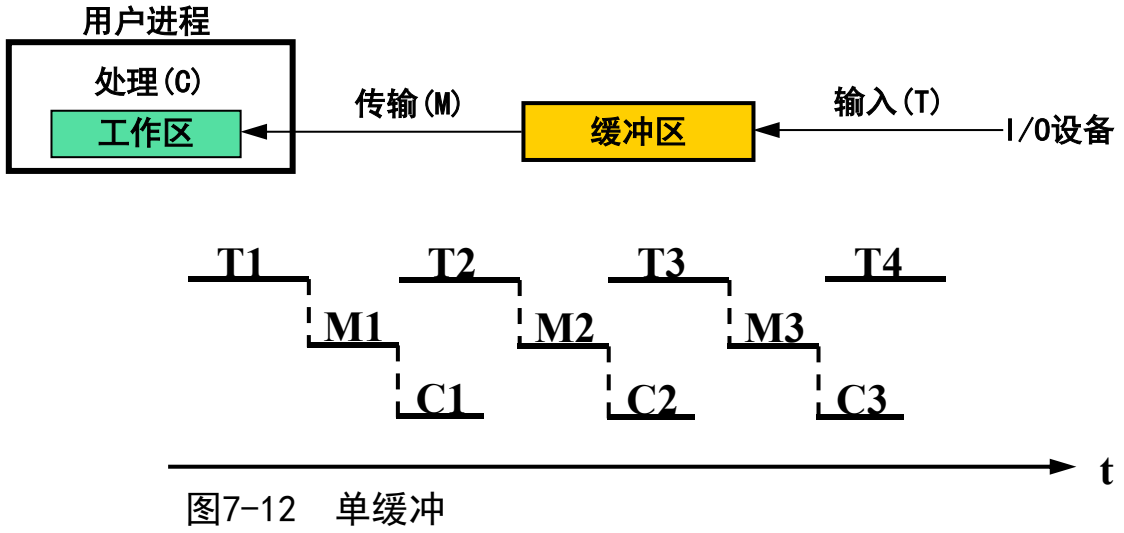


图7-12 单缓冲

- 由于T和C是可以并行的，故系统处理一块数据的时间可表示为 Max(T, C) + M。

7.8 缓冲管理

- **双缓冲**
- 在设备输入时，先将数据送入第一个缓冲区，装满后便转向第二个缓冲区。此时操作系统可以从第一个缓冲区移出数据并送入用户进程，接着CPU对数据进行计算。

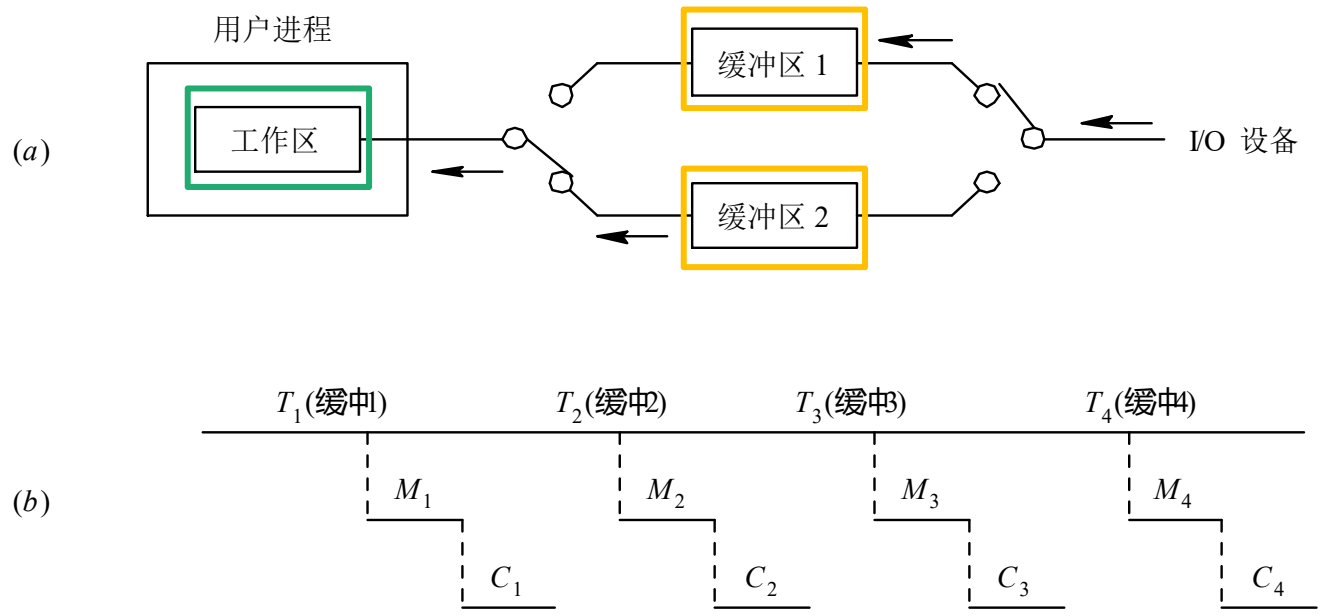
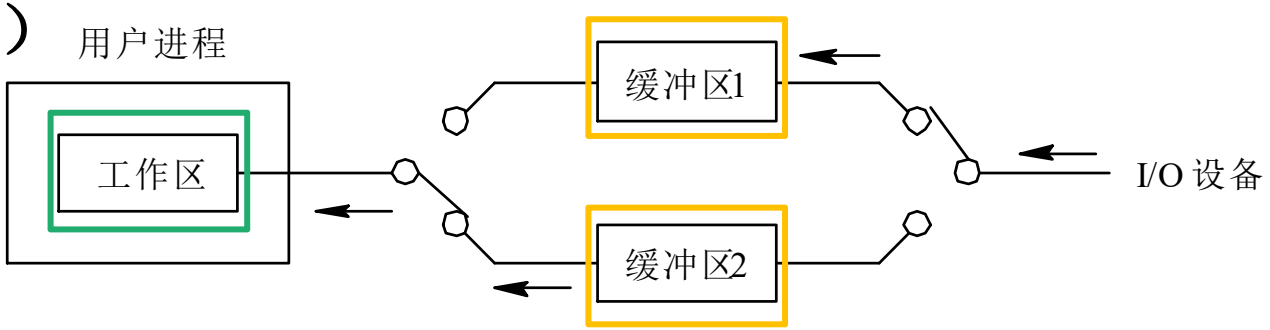


图7-13　双缓冲

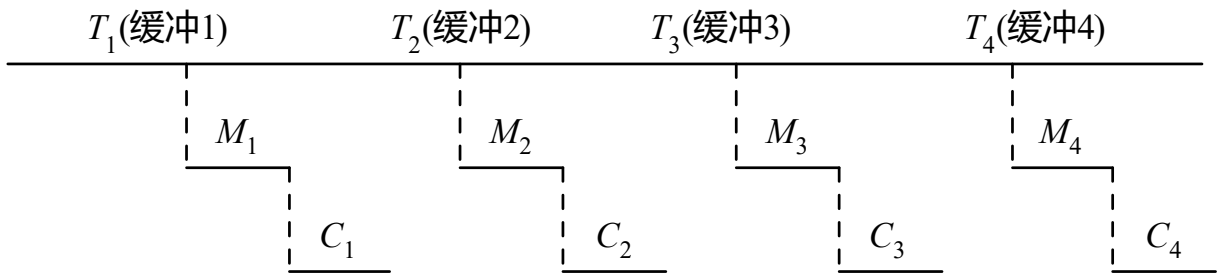
7.8 缓冲管理

- 双缓冲
  - 若C<T，系统处理一块数据的平均时间大约为Max(T,C+M)，若M<<T，则近似为Max(T,C)可使块设备连续输入；
  - 若C>T，系统处理一块数据的平均时间大约为C+M，可使进程不必等待设备（CPU可以不停地计算）

(a)



(b)



7.8 缓冲管理

- 双缓冲
  - 例1：假定从磁盘把一块数据输入到单缓冲区的时间T为2000 μs，将缓冲区中数据传送到用户区的时间M为50μs，而CPU对这块数据处理（计算）时间C为1500μs，则系统对每块数据的处理时间为 2050 μs。如果将单缓冲改为双缓冲，则系统对每块数据的处理时间为 2000 μs。

7.8 缓冲管理

- 双缓冲
  - 例2：某文件占10个磁盘块，现要把该文件磁盘块逐个读入主存缓冲区，并送用户区进行分析。假设一个缓冲区与一个磁盘块大小相同，把一个磁盘块读入缓冲区的时间为100μs，将缓冲区的数据传送到用户区的时间是50μs，CPU对一块数据进行分析的时间为50μs。在单缓冲区和双缓冲区结构下，读入并分析该文件的时间分别是____。

    A．1500μs、1000μs　　B．1550μs、1100μs

    C．1550μs、1550μs　　D．2000μs、2000μs

    B

  - 解答：

    单缓冲(Max(C,T)+M)*10+C=1550

    双缓冲 Max(T，C+M)*10+(C+M)=1100

7.8 缓冲管理

- 循环缓冲
  - 多个缓冲区
  - 多个指针
  - Getbuf过程和Releasebuf过程

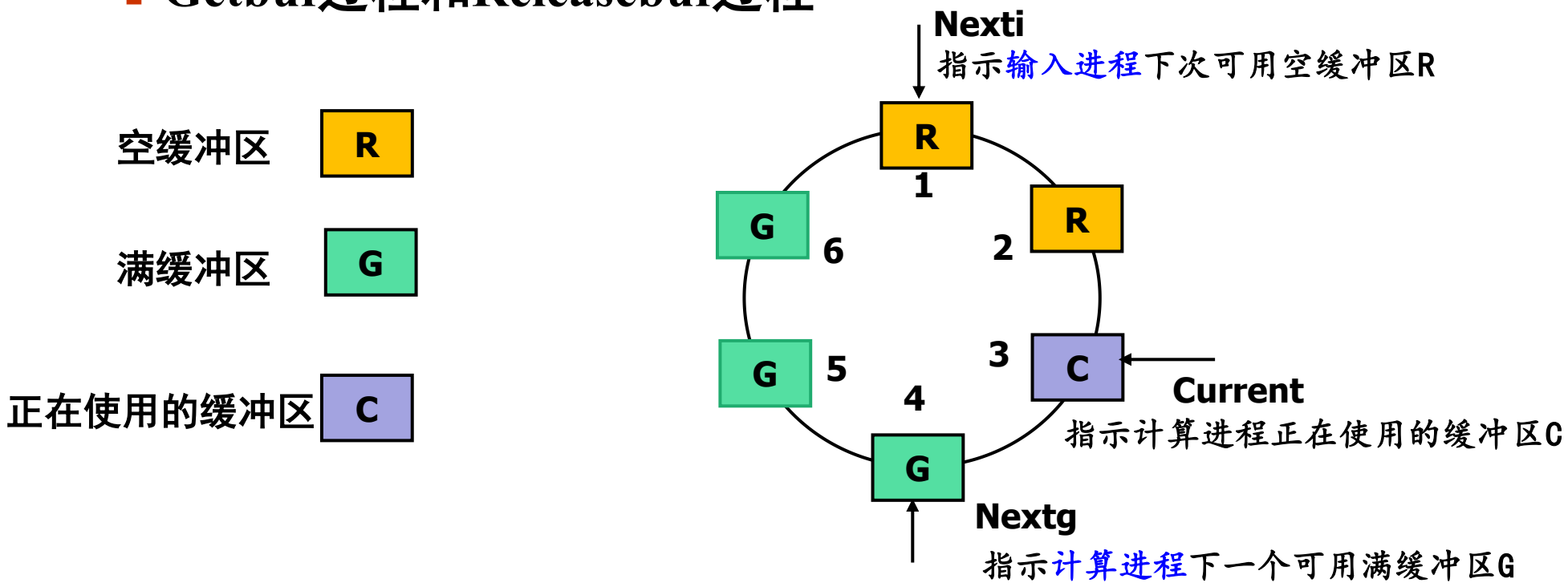


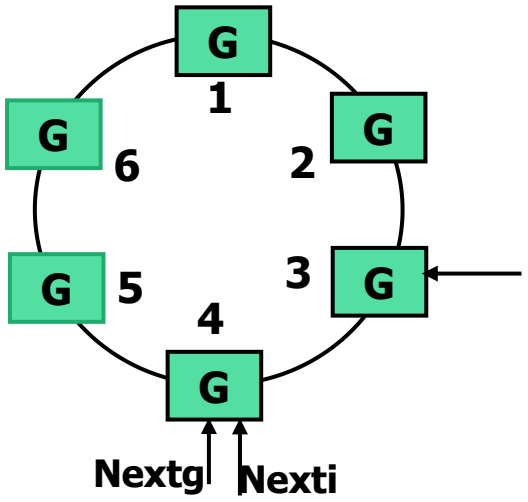
图7-14　循环缓冲

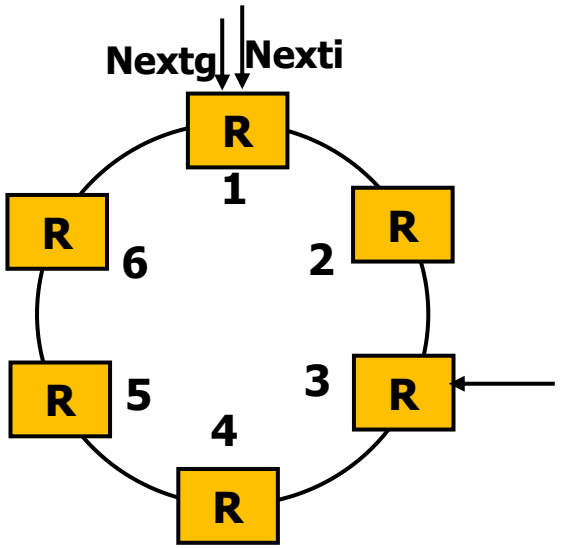
7.8 缓冲管理

- 循环缓冲
  - Getbuf（）过程
    - 计算进程要使用缓冲区时，调用Getbuf过程：将Nextg所指的缓冲区改为“现行”，用Current指向它，同时将Nextg指向下一个满缓冲区G。
    - 输入进程要使用缓冲区时，调用Getbuf过程：将Nexti所指的缓冲区供给输入进程使用，同时将Nexti指向下一个空缓冲区R。
  - Releasebuf（）过程
    - 计算进程把C缓冲区的数据提取完毕时，调用Releasebuf过程，将缓冲区G释放：将该缓冲区由现行缓冲区C改为空缓冲区R。
    - 输入进程把缓冲区装满时，调用Releasebuf过程，将该缓冲区释放，并改为满缓冲区G。

7.8 缓冲管理

- 循环缓冲
  - 进程同步
    - Nexti指针追上Nextg指针。已无空缓冲区，输入进程阻塞，直到计算进程在Releasebuf时唤醒它(输入进程）。
    - Nextg指针追上Nexti指针。计算进程快，已无满缓冲区，计算进程阻塞，直到输入进程用Releasebuf时唤醒它（计算进程）。





7.8 缓冲管理

- **缓冲池**
  - 为了提高缓冲区利用率，目前广泛流行缓冲池，在池中设置了多个可供若干进程共享的缓冲区。
  - **缓冲池与缓冲区的区别：缓冲区是一组内存块的链表，缓冲池包括相应的的数据结构和一组操作函数，以管理多个缓冲区。**

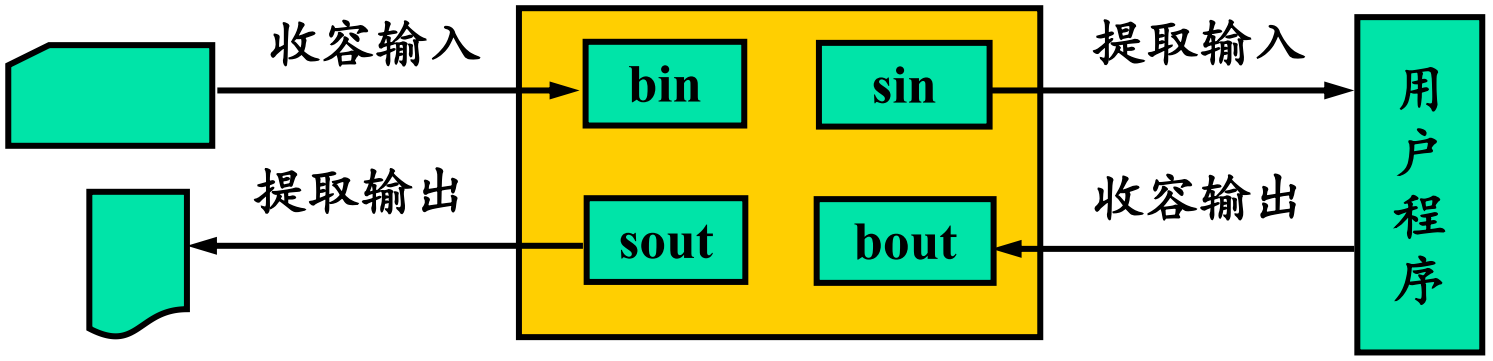
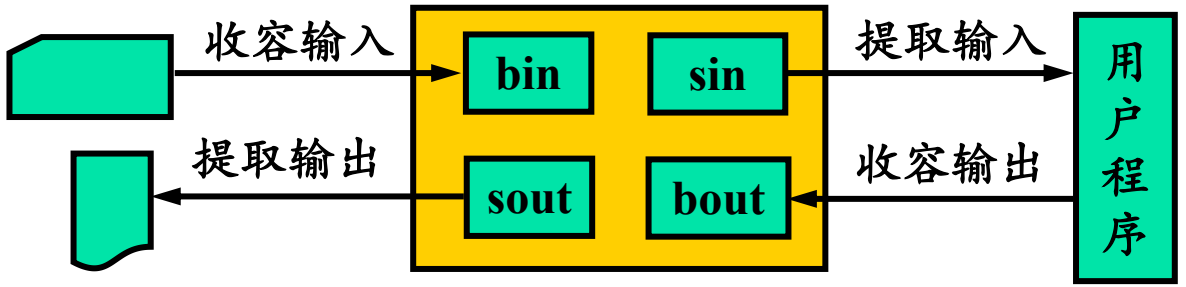


图7-15　缓冲池

7.8 缓冲管理

- 缓冲池的组成
  - 四种工作缓冲区：bin, sin, sout, bout
  - 三个队列:空缓冲区队列emq;装满输入数据的缓冲区队列inq；装满输出数据的缓冲区队列outq;
  - 两个过程：getbuf过程和putbuf过程
  - 四种工作方式:收容输入；提取输入；收容输出；提取输出；



  - bin:用于收容输入数据　　sin:用于提取输入数据
  - bout:用于收容输出数据　　sout:用于提取输出数据

7.8 缓冲管理

- 缓冲池的组成
  - Getbuf和Putbuf过程
  - 设type指向队列，互斥信号量MS(type)，资源信号量RS(type)

  - Procedure Getbuf(type) //从type所指队列的队首摘取一个缓冲区

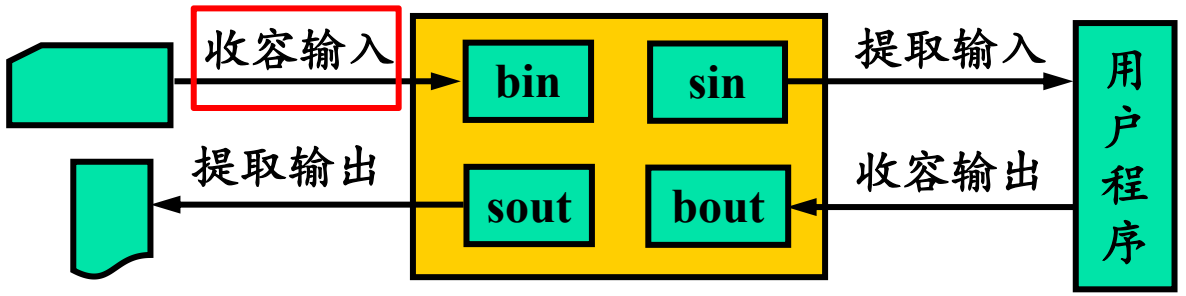
```
{ wait(RS(type));
  wait(MS(type));
  B(number)=Takebuf(type);//从type所指队列的队首摘取一个缓冲区
  signal(MS(type));
}
```

  - Procedure Putbuf(type， number) //将number所指缓冲区挂到type所指队列上

```
{
 wait(MS(type)) ;
 Addbuf(type,number); //将number所指缓冲区挂到type所指队列上
 signal(MS(type)) ;
 signal(RS(type)) ;
}
```

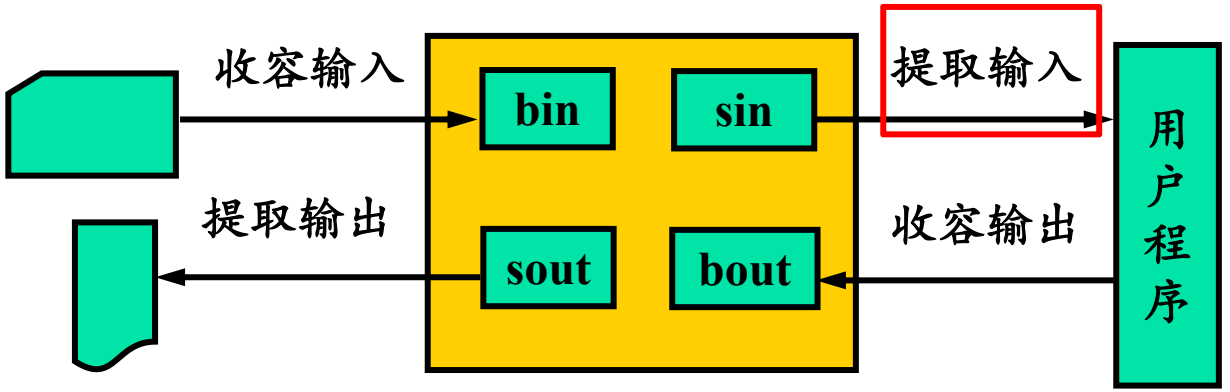
7.8 缓冲管理

- 缓冲池的组成
  - 收容输入
  - 调用Getbuf(emq)过程，从空缓冲队列emq的队首摘下一个空缓冲区，把它作为收容输入工作缓冲区bin。
  - 然后将数据输入到bin。
  - 装满后调用Putbuf(inq，bin)过程，将该缓冲区bin挂到输入队列inq上。



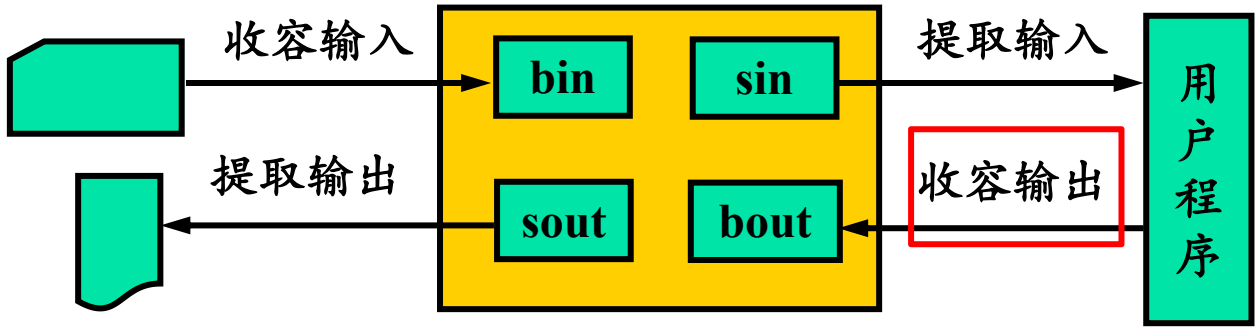
7.8 缓冲管理

- 缓冲池的组成
  - 提取输入
  - 计算进程需要数据时，调用Getbuf(inq)过程，从输入缓冲队列inq的队首摘下一个缓冲区，作为提取输入工作缓冲区sin，
  - 计算进程从sin中提取数据。
  - 计算进程用完数据后，再调用Putbuf(emq，sin)过程，将该缓冲区sin挂到空缓冲队列emq上。



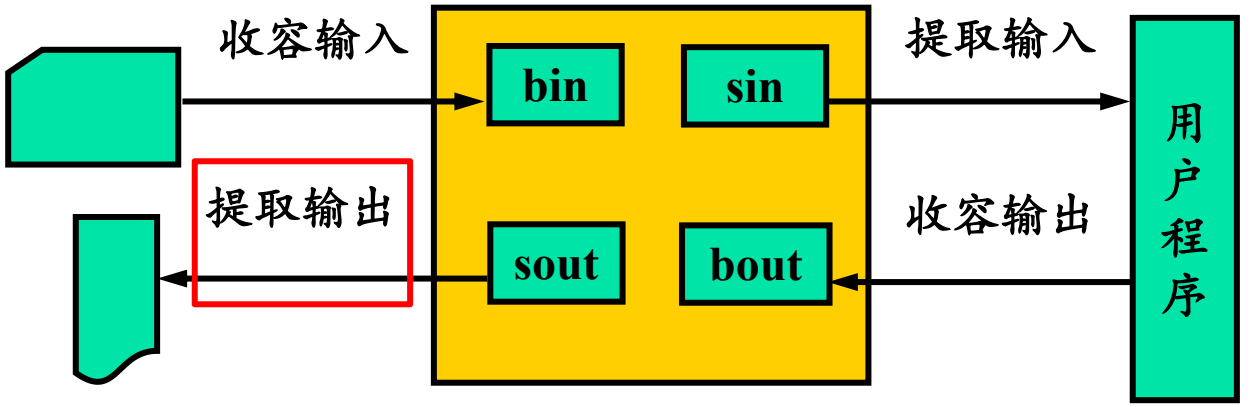
7.8 缓冲管理

- 缓冲池的组成
  - 收容输出
  - 当计算进程需要输出时，调用Getbuf(emq)过程，从空缓冲队列emq的队首摘下一个空缓冲区，把它作为收容输出工作缓冲区bout。
  - 当其中装满输出数据后，调用Putbuf(outq，bout)过程，将该缓冲区挂到输出队列outq上。



7.8 缓冲管理

- 缓冲池的组成
  - 提取输出
  - 由输出进程调用Getbuf(outq)过程，从输出缓冲队列outq的队首摘下一个缓冲区，把它作为提取输出工作缓冲区sout。
  - 当数据提取完后，再调用Putbuf(emq，sout)过程，将该缓冲区挂到emq队列末尾。



7.9 磁盘存储器的性能和调度

- 磁盘的结构和布局

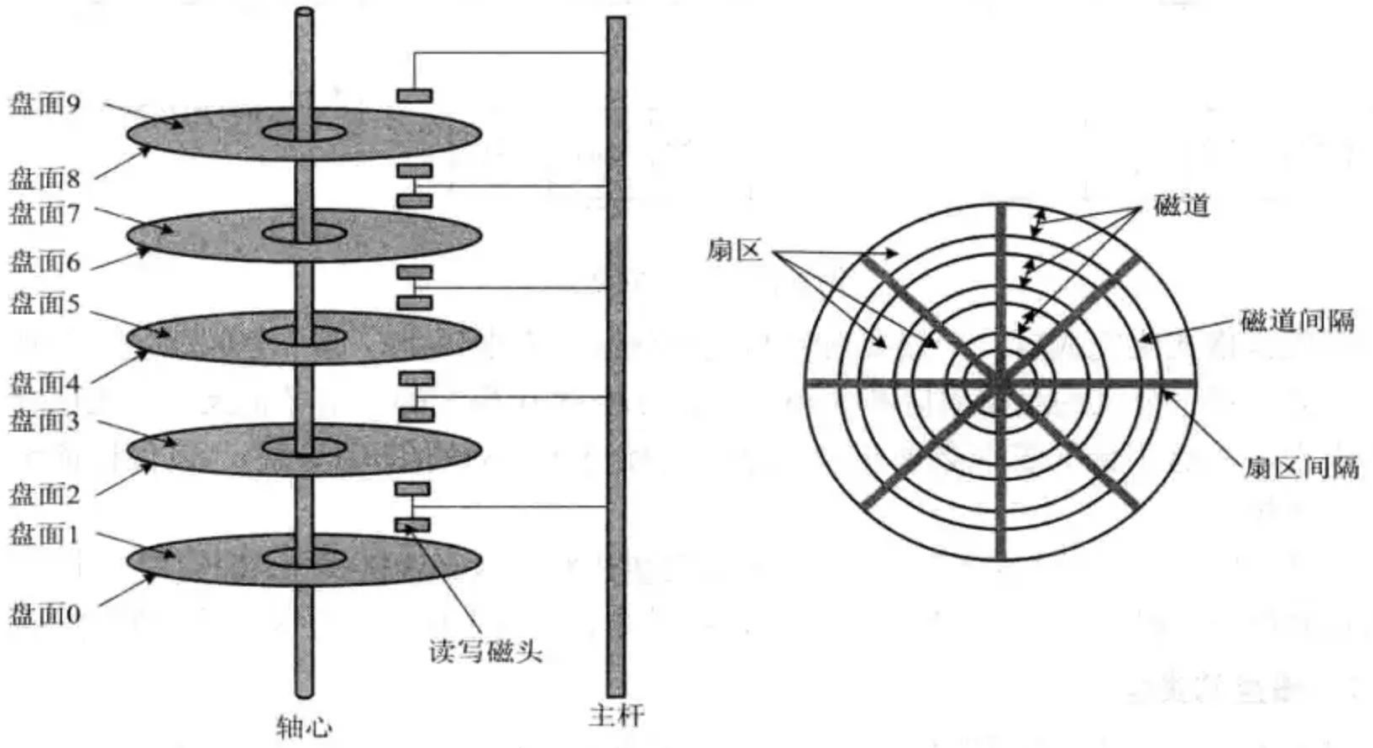


图7-15　磁盘的结构和布局

7.9 磁盘存储器的性能和调度

磁盘格式化与分区

磁盘低级格式化

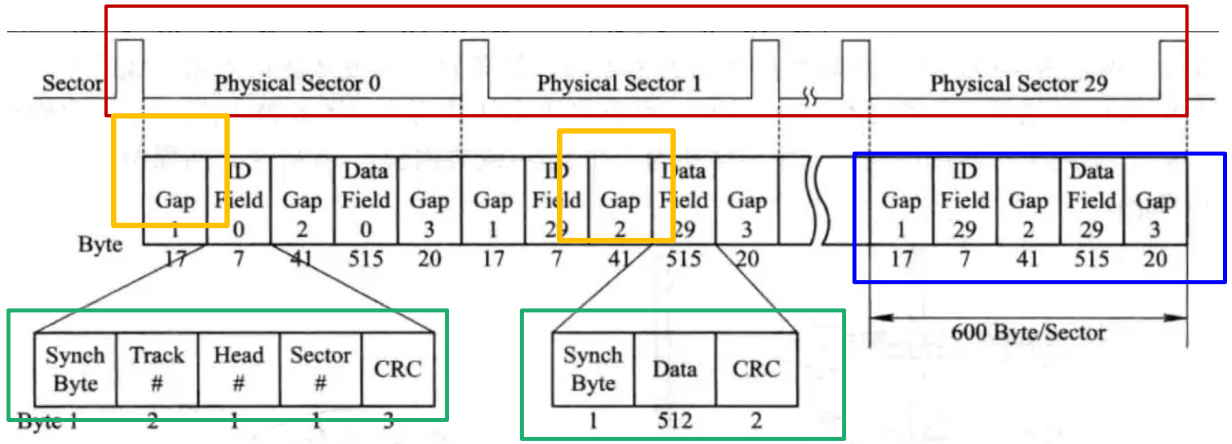


图7-16　一个温盘的一条磁道格式化的情况

- 每条磁道(Track)含有30个固定大小的扇区(Sectors)
- 每个扇区容量为600个字节，其中512个字节存放数据，其余的存放控制信息。
- 每个扇区包括两个字段：
- ①标识符字段(ID Field)，其中一个字节的SYNCH具有特定的位图像，作为该字段的定界符，利用磁道号(Track)、磁头号(Head #)及扇区号(Sectors #)三者来标识一个扇区；CRC字段用于段校验。
- ②数据字段(Data Field)，存放512个字节的数据。
- 此外，在磁盘一个盘面的不同磁道(Track)、每个磁道的不同扇区(Sectors)，以及每个扇区的不同字段(Field)之间，为了简化和方便磁头的辨识，都设置了一个到若干个字节不同长度的间距(Gap，简称间隙)。

7.9 磁盘存储器的性能和调度

- 磁盘格式化与分区
  - 分区
    - 在磁盘格式化完成后，一般要对磁盘进行分区。
    - 在逻辑上，每个分区就是一个独立的逻辑磁盘。
    - 每个分区的起始扇区和大小都记录在磁盘0扇区的主引导记录分区表所包含的分区表中。
    - 分区表中必须有一个分区被标记成活动的(即引导块)，以保证能够从硬盘引导系统。

7.9 磁盘存储器的性能和调度

- 磁盘格式化与分区
  - 磁盘高级格式化
    - 磁盘高级格式化是对磁盘分区进行初始化，建立文件系统结构。高级格式化不改变硬盘的物理结构，但会覆盖分区的引导区、文件分配表（FAT）等元数据区域，使其具备存储文件的能力。
    - 磁盘高级格式化的工作原理：根据用户选择的文件系统类型（如FAT32、NTFS、ext4等），在分区中写入特定的文件系统元数据，包括：
      - 引导记录：存储启动加载程序。
      - 文件分配表/索引节点（inode）表：记录文件存储位置。
      - 目录结构：建立根目录和子目录的层次关系。
      - 校验信息：用于数据完整性和错误检测。

7.9 磁盘存储器的性能和调度

- 磁盘格式化与分区
  - 启动装载程序（Bootloader）
  - 功能:启动加载程序是计算机启动时运行的第一个程序，负责加载操作系统内核到内存并启动系统。
  - 常见启动加载程序:
    - Windows：Windows Boot Manager（bootmgr）。
    - Linux：GRUB（Grand Unified Bootloader）或LILO。
    - macOS：Apple Boot ROM + boot.efi。
  - 实现原理:
    - MBR分区表：启动加载程序存储在主引导记录（MBR）的第一个扇区（512字节）。
    - GPT分区表：启动加载程序存储在EFI系统分区（ESP）中，文件名为bootx64.efi（x86_64架构）。

7.9 磁盘存储器的性能和调度

- 操作系统启动
- Windows双系统启动示例：
  - 硬盘分区结构：

```
1  /dev/sda1 (ESP, FAT32) - 启动加载程序（bootmgr）
2  /dev/sda2 (NTFS)       - Windows系统分区
3  /dev/sda3 (ext4)       - Linux系统分区（非引导）
```

  - 启动流程：
    - BIOS/UEFI加载ESP中的bootmgr。
    - bootmgr加载Windows内核并启动系统。
    - 若需启动Linux，用户需通过GRUB（安装在/dev/sda1）引导/dev/sda3。

7.9 磁盘存储器的性能和调度

- 操作系统启动
- Linux单系统启动示例：
  - 硬盘分区结构：

```
1  /dev/sda1 (ESP, FAT32) - GRUB启动加载程序
2  /dev/sda2 (ext4)       - Linux根分区（引导分区）
3  /dev/sda3 (ext4)       - /home分区（非引导）
```

  - 启动流程：
    - UEFI加载ESP中的GRUB。
    - GRUB加载/dev/sda2中的Linux内核并启动系统。
    - /dev/sda3中的用户数据在系统启动后被访问。

7.9 磁盘存储器的性能和调度

- 操作系统启动
- 数据备份分区示例：
  - 硬盘分区结构：

```
1   /dev/sda1 (NTFS) - Windows系统分区（引导）
2   /dev/sda2 (NTFS) - 数据备份分区（非引导）
```

  - 启动流程：
    - BIOS加载/dev/sda1中的bootmgr。
    - Windows启动后，用户可访问/dev/sda2中的备份数据。

7.9 磁盘存储器的性能和调度

- 磁盘的类型
  - 单片盘和多片盘
  - 固定头磁盘和活动头磁盘
    - 固定头磁盘：每条磁道上都有一读/写磁头，可以进行并行读/写。
    - 移动头磁盘：每个盘面仅配有一个磁头，也被装入磁臂中。移动磁头仅能以串行方式读写，致使其I/O速度较慢

7.9 磁盘存储器的性能和调度

磁盘的访问时间

- 磁盘访问时间T_a包括以下三部分时间：

（1）寻道时间T_s是指把磁臂（磁头）移动到指定磁道上所经历的时间。

（2）旋转延迟时间T_τ是将指定扇区移动到磁头下面所经历的时间。

（3）传输时间T_t是把数据从磁盘上读出或向磁盘写入数据所经历的时间。

- 磁盘访问时间T_a计算：

$$T_a=T_s+T_\tau+T_t = (m\times n+s)+\frac{1}{2r}+\frac{b}{rN}$$

- 参数说明

m:一个磁道的移动时间

n:移动磁道数

s:启动磁道臂时间

r:每秒的转数

N:一条磁道的字节数

b:传输的字节数

7.9 磁盘存储器的性能和调度

- **磁盘调度**
  - 当有多个进程都要求访问磁盘时，采用某种调度策略，按照某个次序执行进程的磁盘访问请求，这就是磁盘调度。
  - 磁盘调度的目的是减少各进程对磁盘的平均访问时间。
  - 访问时间主要是寻道时间，磁盘调度的目的是使平均寻道时间最少。
  - 目前常用的算法有：
    - 先来先服务
    - 最短寻道时间优先
    - SCAN算法
    - Look算法（改进SCAN算法）
    - 循环扫描(CSCAN)算法
    - NStepSCAN算法
    - FSCAN算法

7.9 磁盘存储器的性能和调度

- **磁盘调度**
  - 先来先服务（FCFS）
  - 例：若递交给磁盘驱动程序的磁盘柱面请求按到达时间顺序分别是55、58、39、18、90、160、150、38、184，设磁头初始处于100柱面，则对于FCFS算法，平均寻道长度为多少？

  - 解：

    服务顺序为：55，58，39，18，90，160，150，38，184

    总寻道长度=45+3+19+21+72+70+10+112+146=498

    平均寻道长度=498/9=55.3

7.9 磁盘存储器的性能和调度

- **磁盘调度**
  - 最短寻道时间优先（SSTF）
  - 例：若递交给磁盘驱动程序的磁盘柱面请求按到达时间顺序分别是55、58、39、18、90、160、150、38、184，设磁头初始处于100柱面，则对于SSTF算法，平均寻道长度为多少？

  - 解：

    服务顺序：90，58，55，39，38，18，150，160，184

    总寻道长度=10+32+3+16+1+20+132+10+24=248

    平均寻道长度=248/9=27.56

    “磁盘粘着”现象

7.9 磁盘存储器的性能和调度

- **磁盘调度**
  - 扫描算法（SCAN算法）
  - SCAN算法要求磁头臂仅仅沿一个方向移动，并在途中满足所有未完成的请求，直到它到达这个方向上的最后一个磁道，然后倒转服务方向，沿相反方向扫描，同样按顺序完成所有请求（电梯调度算法） 。
  - SCAN算法既获得了较好性能，又能防止“饥饿”现象。被广泛应用于大、中、小计算机系统和网络中的磁盘调度。

7.9 磁盘存储器的性能和调度

- 磁盘调度
  - 扫描算法（SCAN算法）
  - 例：若递交给磁盘驱动程序的磁盘柱面请求按到达时间顺序分别是55、58、39、18、90、160、150、38、184，最外磁道是200，设磁头初始处于100柱面，则对于SCAN算法，平均寻道长度为多少？
  - 解：设开始时向磁道号增加方向访问：

    服务顺序：150，160，184，90，58，55，39，38，18

    总寻道长度=50+10+24+(200−18+200−90)+32+3+16+1+20=278

    平均寻道长度=278/9=30.88

    设开始时向磁道号减小方向访问

    服务顺序：90，58，55，39，38，18，150，160，184

    总寻道长度=10+32+3+16+1+20+(18−0+150−0)+10+24=284

    平均寻道长度=284/9=31.55

7.9 磁盘存储器的性能和调度

- 磁盘调度
  - Look算法（改进SCAN算法）
  - 磁头沿当前方向移动，仅处理至最后一个请求后立即反向，无需到达磁盘边缘 。

7.9 磁盘存储器的性能和调度

- 磁盘调度
  - Look算法（改进SCAN算法）
  - 例：若递交给磁盘驱动程序的磁盘柱面请求按到达时间顺序分别是55、58、39、18、90、160、150、38、184，设磁头初始处于100柱面，则对于SCAN算法，平均寻道长度为多少？
  - 解：设开始时向磁道号增加方向访问：

    服务顺序：150，160，184，90，58，55，39，38，18

    总寻道长度=50+10+24+(184−90)+32+3+16+1+20=250

    平均寻道长度=250/9=27.78

    设开始时向磁道号减小方向访问

    服务顺序：90，58，55，39，38，18，150，160，184

    总寻道长度=10+32+3+16+1+20+(150−18)+10+24=248

    平均寻道长度=248/9=27.56

7.9 磁盘存储器的性能和调度

- **磁盘调度**
  - 循环扫描算法（CSCAN）
  - CSCAN算法规定磁头单向扫描，例如只是从外向里扫描，当磁头移动到最里磁道并访问后，立即快速移动到最外的欲访问的磁道，构成循环，进行循环扫描（单向扫描算法）。
  - 采用此算法，进程请求的延迟从2T减为T+Smax，其中T为从里向外或从外向里扫描完要访问的磁道所需的寻道时间，Smax是将磁头从最里磁道迅速返回最外磁道所需时间。

7.9 磁盘存储器的性能和调度

- **磁盘调度**
  - 循环扫描算法（CSCAN）
  - 例：若递交给磁盘驱动程序的磁盘柱面请求按到达时间顺序分别是55、58、39、18、90、160、150、38、184，设磁头初始处于100柱面，则对于CSCAN算法，平均寻道长度为多少？
  - 解：设开始时向磁道号增加方向访问：

    服务顺序：150，160，184，18，38，39，55，58，90

    总寻道长度=50+10+24+166+20+1+16+3+32=322

    平均寻道长度=322/9=35.78

7.9 磁盘存储器的性能和调度

- 磁盘调度
  - N步SCAN扫描算法
  - SSTF、SCAN、CSCAN等算法都可能出现磁臂停留在某处不动的情况（“磁臂粘着”），例如，有一个或几个进程对某一磁道有较高的访问频率，即这个(些)进程对反复请求对某一磁道的I/O操作，从而垄断了整个磁盘设备。
  - N步SCAN算法是将磁盘请求队列分成若干个长度为N的子队列，磁盘调度按FCFS处理这些队列；处理每个队列时按SCAN算法。
  - N很大时，NStepSCAN算法接近SCAN算法；N=1时，N步扫描算法蜕化为FCFS算法。

7.9 磁盘存储器的性能和调度

- **提高磁盘I/O访问速度的途径**
  - 磁盘高速缓存
  - 提前读
  - 延迟写
  - 优化物理块的分布
  - 其他（扩展）

7.9 磁盘存储器的性能和调度

- **提高磁盘I/O访问速度的途径**
  - 磁盘高速缓存是指利用内存中的存储空间，来暂存从磁盘中读出的一系列盘块中的信息。因此，这里的高速缓存是一组在逻辑上属于磁盘， 而物理上是驻留在内存中的盘块。
  - 磁盘高速缓存需要考虑的一些问题：
    - 有磁盘高速缓存后的数据访问与交付
    - 高速缓存内的数据置换策略-访问频率，可预见性
    - 高速缓存区与磁盘区的数据一致性问题
    - 其他

7.9 磁盘存储器的性能和调度

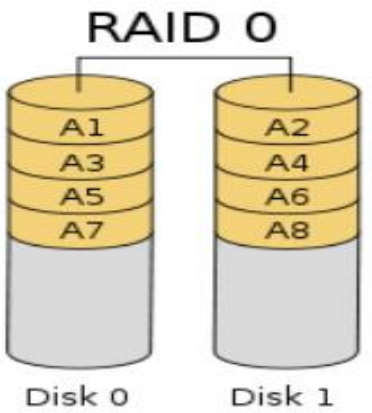
- **提高磁盘I/O访问速度的途径**
  - **提前读：**在顺序访问方式时，在读当前盘块时，将下一个盘块的数据也读入高速缓存中。提前读功能已被广泛采用，如在UNIX系统等。
  - **延迟写：**
    - 缓冲区数据修改后并不立即将缓冲区中数据写入磁盘，而是将它挂在空闲缓冲区队列末尾。
    - 随着空闲缓冲区的使用，缓冲区页缓缓往前移动，直到移到空闲队列之首。（没空闲缓冲区了）
    - 当再有进程申请缓冲区时，才将该缓冲区中的数据写入磁盘，而把该缓冲区作为空闲缓冲区分配出去。
  - **优化物理块的分布：**将同一文件的盘块安排在同一磁道上或相邻磁道上，使磁头的移动距离最小。

7.9 磁盘存储器的性能和调度

- **磁盘冗余交叉系列（RAID）**
  - 1988 年美国加州大学伯克利分校的 D. A. Patterson 教授等首次在论文“A Case of Redundant Array of Inexpensive Disks” 中提出了 RAID 概念，即廉价冗余磁盘阵列（ Redundant Array of Inexpensive Disks）。
  - RAID的基本思想是将多个容量较小、相对廉价的磁盘进行有机组合，从而以较低的成本获得与昂贵大容量磁盘相当的容量、性能、可靠性。
  - RAID分级：RAID 0-RAID6
    RAID 7
    RAID组合等级，RAID01 和 RAID10

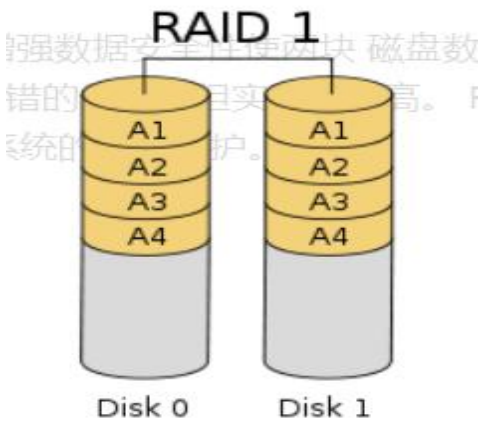
7.9 磁盘存储器的性能和调度

- 磁盘冗余交叉系列（RAID）
- RAID0（条带化）
  - **原理**：将数据均匀分布在多个磁盘上，实现并行读写。
  - **优点**：性能提升——读写速度显著提升，适合需要高速数据传输的应用。
  - **缺点**：无冗余——任何一块磁盘故障都会导致数据丢失。
  - **适用场景**：临时数据存储、视频编辑等对性能要求高但对数据安全性要求低的场景。



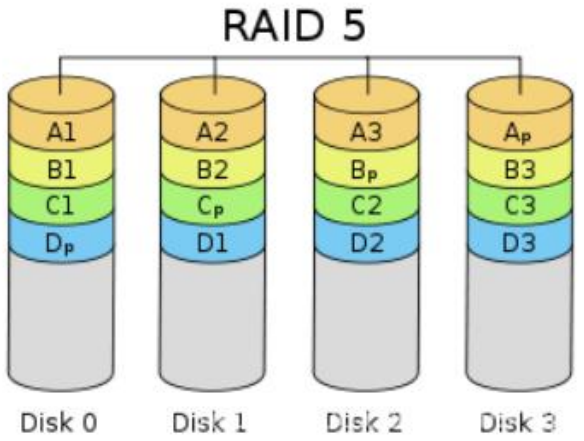
7.9 磁盘存储器的性能和调度

- 磁盘冗余交叉系列（RAID）
- RAID1（镜像）
  - **原理**：将数据完全复制到两个或多个磁盘上，实现冗余备份。
  - **优点**：高可靠性：即使一块磁盘故障，数据仍然可用。
  - **缺点**：成本高：存储效率仅为50%。
  - **适用场景**：需要高数据安全性的关键业务系统。



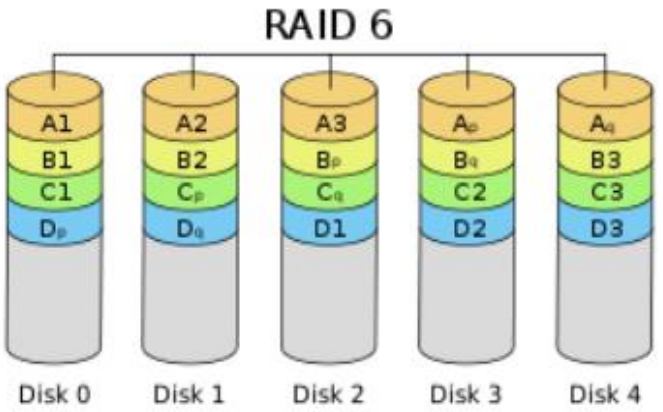
7.9 磁盘存储器的性能和调度

- 磁盘冗余交叉系列（RAID）
- RAID5（带奇偶校验的条带化）
  - **原理**：将数据和奇偶校验信息分布在多个磁盘上，允许一块磁盘故障而不丢失数据。
  - **优点**：性能与冗余的平衡：读写性能较好，且提供数据冗余。
  - **缺点**：重建时间长：磁盘故障后，重建过程可能影响性能。
    写入性能下降：每次写入需要计算奇偶校验信息。
  - **适用场景**：中小型企业服务器、文件服务器等。



7.9 磁盘存储器的性能和调度

- 磁盘冗余交叉系列（RAID）
- RAID6（双重奇偶校验的条带化）
  - **原理**：与RAID 5类似，但使用双重奇偶校验，允许两块磁盘故障而不丢失数据。
  - **优点**：更高的可靠性——支持两块磁盘同时故障。
  - **缺点**：更高的成本——需要更多的磁盘空间存储奇偶校验信息。
    写入性能更低——双重奇偶校验计算增加了开销。
  - **适用场景**：对数据安全性要求极高的关键业务系统。



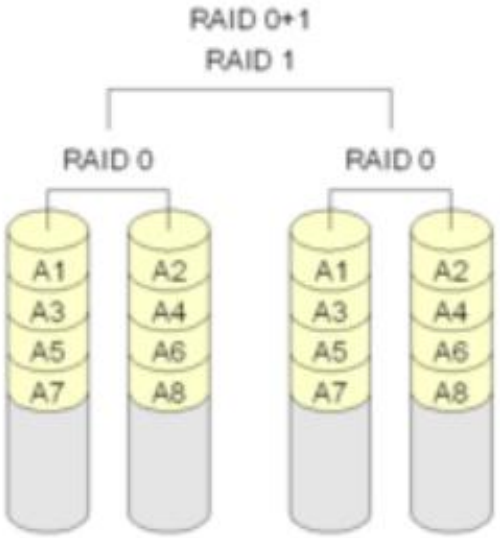
7.9 磁盘存储器的性能和调度

- 磁盘冗余交叉系列（RAID）
- RAID7
  - RAID7 是一种理论上提出的高性能磁盘阵列技术，但并未被广泛标准化或商业化。其概念由 Storage Computer Corporation 提出，旨在通过高度集成化和智能化的设计，提供比传统 RAID 级别更优的 I/O 性能和数据传输速率。
  - RAID7 的核心设计思想
    - 独立存储系统
    - 异步 I/O 和高速缓存
    - 智能操作系统
    - 高扩展性和冗余性

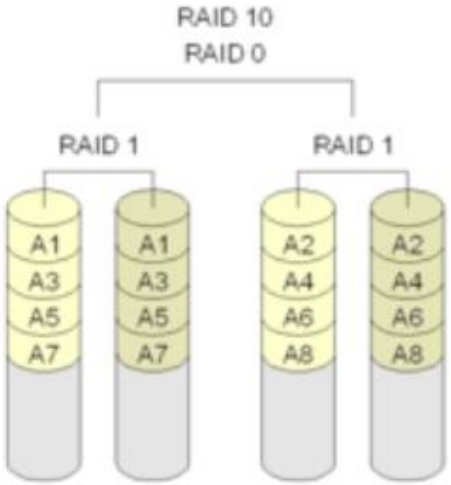
    （课外有兴趣了解下）

7.9 磁盘存储器的性能和调度

- 磁盘冗余交叉系列（RAID）
- RAID组合等级



RAID01：先镜像再条带



RAID10：先条带再镜像